Optimizing Performance and Efficiency in Large Language Models through Dynamic Hybrid Framework

by

Faiyaz Bin Yousuf
22101845
Nahid Hassan
22101822
Asief Iqbal Dieyaz
23341081

A thesis submitted to the Department of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
B.Sc. in Computer Science and Engineering

Department of Computer Science and Engineering
Brac University
June 2025

© 2025. Brac University
All rights reserved.

Declaration

It is hereby declared that

1. The thesis submitted is my/our own original work while completing degree at Brac University.
2. The thesis does not contain material previously published or written by a third party, except where this is appropriately cited through full and accurate referencing.
3. The thesis does not contain material which has been accepted, or submitted, for any other degree or diploma at a university or other institution.
4. We have acknowledged all main sources of help.

Student's Full Name & Signature:

Faiyaz Bin Yousuf
22101845

Asief Iqbal Dieyaz
23341081

Nahid Hassan
22101822

Approval

The thesis/project titled "Optimizing Performance and Efficiency in Large Language Models through Dynamic Hybrid Framework" submitted by

1. Faiyaz Bin Yousuf (22101845)
2. Asief Iqbal Dieyaz (23341081)
3. Nahid Hassan (22101822)

of Summer, 2025 has been accepted as satisfactory in partial fulfillment of the requirement for the degree of B.Sc. in Computer Science in 19 June, 2025.

Examining Committee:

Supervisor:
(Member)

Muhammad Iqbal Hossain, PhD

Associate Professor
Department of Computer Science and Engineering
Brac University

Thesis Coordinator:
(Member)

Dr. Md. Golam Rabiul Alam

Professor
Department of Computer Science and Engineering
Brac University

Head of Department:
(Chair)

Sadia Hamid Kazi, PhD

Chairperson and Associate Professor
Department of Computer Science and Engineering
Brac University

Abstract

The remarkable rise of Large Language Models(LLMs) has made a significant paradigm shift in the field of Artificial Intelligence. LLMs have shown amazing capabilities in a wide range of Natural Language Processing (NLP) tasks. However, the computation cost and memory requirement of the models are high, making them quite resource-intensive. This is a major obstacle to their deployment in resource-constrained environments or platforms, such as local servers, mobile devices, IoT devices, or other edge devices. The existing suite of compression techniques, like knowledge distillation, Low-Rank Adaptation(LoRA), pruning, and quantization, has all proven effective in reducing the computational and memory overhead of LLMs. However, most of them remain limited to static methods, applying a fixed compression level that fails to adapt to dynamic changes in available memory, latency requirements, or prompt complexity. This study aims to develop a novel, resource-aware hybrid framework that dynamically selects the optimal combination of compression techniques during inference. This involves developing a reinforcement learning controller agent and integrating it with state-of-the-art pruning and quantization methods. Afterwards, the dynamic hybrid framework will be evaluated against static baselines on standard benchmarks, assessing accuracy, inference speed, and memory footprint. The expected contribution is a practical, adaptive, and sustainable solution that enables the broader deployment of powerful LLMs on everyday devices.

Keywords: Large Language Models (LLMs), Hybrid Compression, Dynamic Optimization, Adaptive Systems, Pruning, Quantization, Edge Devices, Resource-Aware Inference, Adaptive Controller Agent,Reinforcement Learning

Acknowledgement

First and foremost, all praise and gratitude are due to the Almighty Allah for providing us with the strength, knowledge, and perseverance to complete our Thesis paper successfully. We would also like to express our gratitude to our supervisor, Muhammad Iqbal Hossain, sir, for giving us the best level of guidelines and support for our successful completion. His constant guidance and encouragement have been the perfect motivation for us to refine our ideas and put ample effort into the paper. Lastly, we are very much thankful to our parents, families, colleagues, and friends who have constantly supported us in fulfilling our goals. May Allah bless them all.

Table of Contents

Declaration **i**

Approval **ii**

Abstract **iii**

Acknowledgment **iv**

Table of Contents **v**

List of Figures **vii**

Nomenclature **ix**

1 Introduction **1**
1.1 Background 1
1.2 Rationale of the Study or Motivation 1
1.3 Problem Statement 2
1.4 Objective 3
1.5 Methodology in Brief 4
1.6 Scopes and Challenges 5

2 Literature Review **7**
2.1 Preliminaries 7
2.2 Review of Existing Research 8
2.3 Summary of Key Findings 24

3 Impacts and Constraints **26**
3.1 Societal Impact 26
3.2 Environmental Impact 26
3.3 Project Management Plan 27

4 Proposed Methodology **28**
4.1 Design Process or Methodology Overview 28
4.1.1 Research Workflow 28
4.1.2 Dataset Design and Construction 28
4.1.3 Prompt Curation Pipeline 32
4.1.4 Prompt Complexity Scoring 33
4.1.5 Dataset Validation 35
4.1.6 Baseline Pruner Evaluation 39
4.1.7 Result Analysis and Discussion 41
4.2 Preliminary Design and Specification 43
4.2.1 Custom Architecture Overview 44
4.2.2 Details of Implementation 46
4.2.3 Result Analysis 49
4.3 Future Work 53

5 Conclusion **54**

Bibliography **55**

Appendix A: Statistical Validation Tables **59**

List of Figures

1.1 Methodology Roadmap 4

2.1 A taxonomy of techniques for achieving resource-efficient LLMs. (Bai et al.) 9

4.1 Research Workflow Methodology 28
4.2 Distribution of categories of the dataset 30
4.3 Distribution of subjects of the dataset 30
4.4 Complexity Score Distribution 31
4.5 Complexity by Context Dependency 32
4.6 Prompt Curation Pipeline 32
4.7 Prompt Complexity Score 34
4.8 SparseGPT Perplexity Comparison 41
4.9 LLM-Pruner Perplexity Comparison 42
4.10 SliceGPT Perplexity Comparison 42
4.11 System Architecture 45
4.12 Training Workflow Diagram 47
4.13 Inference Time During Training 50
4.14 Perplexity During Training 50
4.15 Inference Time During Testing 51
4.16 Perplexity During Testing 52

Nomenclature

The next list describes several symbols & abbreviation that will be later used within the body of the document

API Application Programming Interface

AWQ Activation-aware Weight Quantization

$BOPs$ Bit-Operations

CPU Central Processing Unit

DIP Dynamic Input Pruning

$DJPQ$ Differentiable Joint Pruning and Quantization

DQN Deep Q-Network

FFN Feed-Forward Network

$FLOPs$ Floating Point Operations per second

GPU Graphics Processing Unit

IoT Internet of Things

$KV - cache$ Key-Value cache

LLM Large Language Models

$LoRA$ Low-Rank Adaptation

LWC Learnable Weight Clipping

MAC Multiply-And-Accumulate

$MMLU$ Massive Multitask Language Understanding

MoE Mixture of Experts

$NF4$ 4-bit NormalFloat

NLP Natural Language Processing

PAT Pruning-Aware Tuning

PPO Proximal Policy Optimization

PTQ Post-Training Quantization

QAT Quantization-Aware Training

$QLORA$ Quantized Low-Rank Adaptation

RAP Runtime-Adaptive Pruning

RL Reinforcement Learning

RTN Round-to-Nearest

SDG Sustainable Development Goals

$W8A8$ 8-bit Weight and 8-bit Activation quantization

Chapter 1
Introduction

1.1 Background

In recent years, Large Language Models (LLMs) have made significant advancements in the field of artificial intelligence. Open-source LLMs such as BERT, Stable Diffusion, Llama, etc have created opportunities, including high-end text analytics and code efficiency, to the design of digital art. Such models are constructed on elaborate designs of billions of parameters, which enable them to comprehend and impressively produce human-like content with remarkable accuracy. The issue is that such LLMs need immense amounts of computational power and intensive resources. They typically require advanced processing tools, storage, etc., and high-end memory, which is not affordable to most people with standard devices. When a person wishes to run an LLM on a weaker machine, they must manually tweak its settings. This involves a lot of trial and error, adjusting things like model size through pruning or reducing the precision of its calculations through quantization. This process is time-consuming and requires a lot of technical expertise. Due to such difficulties, a widening disparity exists between the capabilities of these LLMs and the people capable of utilizing them. This weakness hinders the spread of AI in real-life use and blocks many developers and scientists from being at liberty with these formidable weapons.

1.2 Rationale of the Study or Motivation

The main motivation behind this research is to break down the hardware barriers that currently limit the use of LLMs [4]. Right now, the way we deploy these models is largely static; a model is released with a single configuration that assumes the user has powerful hardware. This "one-size-fits-all" approach is inefficient and exclusive to specific hardware and resources. It's similar to a modern video game being released with only "Ultra" graphics settings, making it unplayable on most standard computers. In modern video games, developers solve this by offering a range of graphics settings (low, medium, high) and often an "auto-detect" feature that configures the game according to the device the game is being played. This is precisely the kind of solution that LLMs require. This study proposes to develop an adaptive system that infers resource-aware auto-detection settings for large language models. The goal is to create a compression framework that can first analyze the user's hardware—identifying the type of processing units, the amount of available memory, storage, and the capabilities of the processing units—and then automatically adjust the LLM's parameters for the best possible performance on that device. It's not just about making the model run; it's about making it run optimally. This means finding the perfect balance between speed and accuracy for that specific hardware, ensuring the user gets the best experience their device can offer. By creating such a system, it can democratize the use of LLMs for all groups of people. Students could run sophisticated models on their personal laptops for research, developers in smaller companies could integrate AI features into their mobile apps without

worrying about performance issues, and innovators in resource-constrained environments could experiment with LLMs more efficiently. As AI is becoming part of our daily lives, it needs to be more inclusive and efficient, moving away from a static deployment model to a dynamic one where powerful tools adapt to the user, not the other way around.

1.3 Problem Statement

The promotional spread of LLM as the component blocks of generative AI, a wider evaluation and application is restricted due to the memory and processing demands of LLM, which are impractical to realize on an individual computer. This is mostly with respect to the world of limited resources i.e. through personal computers, mobile devices, edge sensors, local servers etc. which cannot accommodate the high end GPUs. The primary concern is to take those strong patterns further and their application to the context in which computationally demanding challenges cannot be performed. The first actual effort to address this problem has been clouding down to static compression techniques, which include pruning, quantization, knowledge distillation, and so on. The techniques reduce the model permanently, including its strength in advance of deployment. Though it is good to minimise the model footprint, this non-adjustable single size fits all approach, by definition, is inflexible. It fails to adapt to the dynamic environment which is uncertain in the true working situations. As an illustration, the memory of a device that is available, processing power and battery power might differ significantly, and the complexity of user prompts. Model used is statically compressed and will fail to optimize to such changes and may fail to operate efficiently and consume too much energy which may later fail in due course should resources be overstretched. It has led to the need to seek hybrid compression schemes, which is a combination of methods like pruning and quantization. The hybrid methods are however not dynamic: before, a compression strategy is fixed, and the same strategy is applied during the inference. This comes at a tradeoff that does not entail a major opportunity to maximize. The outstanding critical matter is in no dynamic frame work to change the degree and type of compression on the fly based on the temporary availability of resources in the machine, along with those of the subsequent job. More recent research has never taken into account systems that explicitly integrate the real-time and resources-aware decision-making process into the compression pipeline. Such a system contains the elements, among others, of pruning such networks with algorithms, including the popular ones, e.g., Wanda, and quantisation of the weights to fit into a given number of bits, e.g., GPTQ. A combination of the various elements under a dynamic control mechanism is what has not been succinctly achieved. This is a large research gap to come up with an intelligent adaptive framework that is capable of selecting the optimal compression policy in a resource-constrained and dynamic environment. One such system would then be aggressive pruning and quantization when the battery is low. The user prompt display would be in a less compressed form when the device is being charged up and the job being done by the user requires the greatest level of precision. Therefore, the key point is that the LLMs are still unable to manage themselves when it comes to using resources based on the ever-changing environmental conditions and given tasks. A dynamic compression framework is not only a step towards the right direction. Nevertheless, it is a necessary modification to make sure that LLMs can be carried and can be made sustainable. It can ensure the expansion of the range of the potent AI in low-resource contexts, such as NGOs or offline infrastructures, and the possibilities of

even greater digital inclusivity and allow a complete new set of intelligent applications, that are more powerful but less resource-consuming, to be introduced to the world.

1.4 Objective

Also relying on the current advances in the sphere, this study will work on the new framework of dynamic, hybrid compression of LLMs, which will be able to adjust to the alterations of a real-time environment. Prior art in LLM compression is static, i.e., using a given amount of optimization that does not consider the deployment environment's volatile resources, such as mobile devices or edge servers. It is not efficient when it comes to this so-called one-size-fits-all method. This study closes this essential gap by creating a system capable of using feedback at the hardware level and the complexity of the user prompt to change its strategy on compression dynamically.
The central point of this research is to develop a lightweight Reinforcement Learning(RL)-driven Controller Agent to keep track of the device statistics (CPU load, GPU utilization, free memory, and battery level). The controller will, in turn, choose and implement the most suitable set of compression methods. It will select the most appropriate pruning configuration in real-time choosing components such as attention heads, KV caches, feed-forward neurons, and transformer layers in response to these dynamic inputs. This will lead to a temporarily optimal LLM dedicated to the particular task and the present state of the device, thus making the inference efficient with barely noticeable accuracy drop. Building on top of existing open-source frameworks and using existing compression libraries, this study will create empirical insight into the trade-offs of computational resource consumption and quality of models. The end objective would be to develop a modular, scalable, and sustainable dynamically hybrid framework system that ensures that advanced deployment of LLM would be viable and sustainable across a broader scope of devices, especially in resource-constrained environments. This research will add a realizable and flexible solution to a prominent issue of the operationalization of large-scale AI.
Research Objectives of this research include

1. Develop a lightweight, RL-based controller agent that continuously profiles device telemetry (CPU utilization, memory availability, GPU load) alongside incoming prompt complexity metrics to inform compression decisions.

2. Create an integrated and flexible hybrid compression module, combining adaptive structured pruning that integrates attention-head, feed-forward, transformer layer, and KV-cache pruning with rapid-switch orchestration at inference time.

3. Implement adaptive inference protocols that modulate compression levels based on resource availability (including memory usage, latency targets, and energy constraints) within acceptable accuracy bounds.

4. Design and execute comprehensive benchmarking under controlled simulations of extreme device constraints to evaluate end-to-end throughput and quality-of-service.

5. Train and validate the RL agent that will learn optimal pruning strategies balanc-

ing inference speed, perplexity, and accuracy.

6. Perform rigorous multi-factor analysis to quantify the frontiers between compression aggressiveness and model performance metrics, including perplexity, accuracy, inference speed, and energy consumption.

1.5 Methodology in Brief

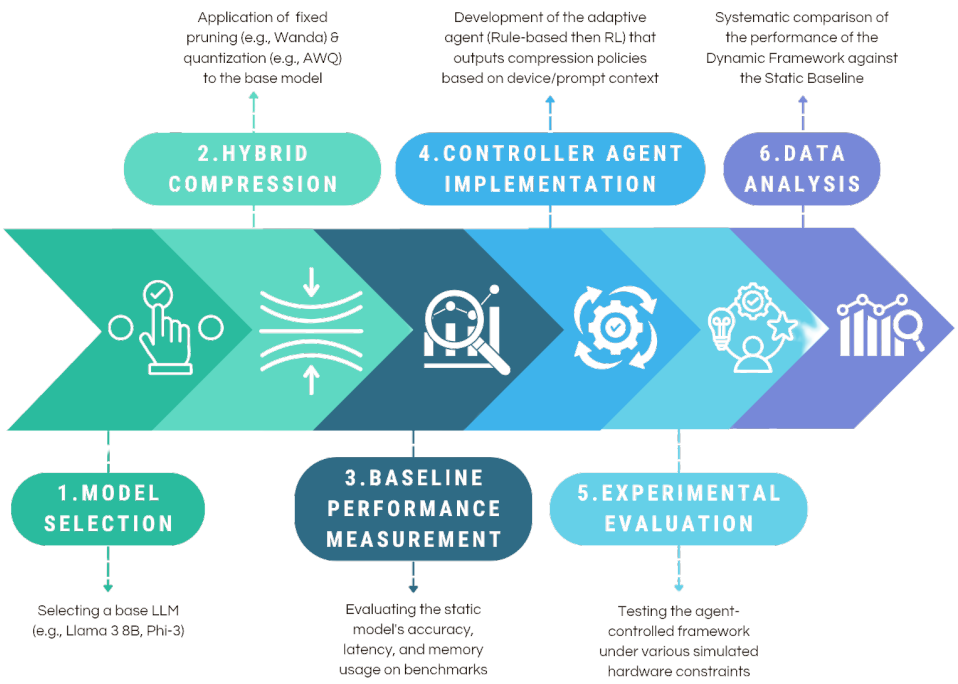

Figure 1.1: Methodology Roadmap

The study employs state-of-the-art hybrid model compression techniques, accompanied by a controlling agent to improve the efficiency of LLMs using an adaptive and dynamic framework. The research methodology is structured and systematically illustrated in the diagram above. This approach is intended to ensure a logical progression from establishing the foundational benchmarks to implementing the core dynamic components, concluding with a comprehensive comparative evaluation. The methodology is as follows:

1. **Model Selection:** A medium-scale open-source LLM like LLaMA-3(1B) will be selected as the base architecture. They will be chosen based on their open availability, strong performance, and suitability for running in resource-limited environments.

2. **Baseline Static Compression:** A standard, state-of-the-art static hybrid compression pipeline will be implemented. In this process, a fixed level of pruning will be applied to the base model. For example, we will use a one-shot, activation-aware pruning method like Wanda [27] or SparseGPT [20] to achieve a pre-defined sparsity (e.g., 50%).

3. **Baseline Performance Measurement:** The performance of the static hybrid compression model will be rigorously evaluated on a suite of standard benchmarks, including reasoning (GSM8k, MMLU) and instruction-following (OpenOrca) tasks. Key metrics to be recorded will be accuracy, latency of the inference, and peak memory footprint in the final phase. Currently, we are using our mock dataset to understand the performance.

4. **Controller Agent Implementation:** The Reinforcement Learning Controller agent, regarded as the brain of the framework, will be developed. Designed to receive real-time context as input, including device statistics (available RAM, CPU load, etc) and prompt characteristics (e.g, token count, complexity heuristics, perplexity)– and give an optimal compression policy as output for that specific inference request. Initially, we will start with a rule-based heuristic controller based on a set of logical rules, such as checking prompt length and RAM, and then applying the fixed pruning ratios and quant bits for that particular condition. Afterward, we will extend this by using a Reinforcement Learning (RL) controller agent (e.g, a lightweight PPO or DQN algorithm), following the paradigm set by frameworks like RAP (Runtime-Adaptive Pruning for llm inference). The agent will learn a comprehensive policy by optimizing a reward function, balancing both inference speed, accuracy, and resource consumption.

5. **Experimental Evaluation:** Various resource-constrained environments will be simulated, which involve programmatically setting budgets for memory usage and simulating different levels of CPU or GPU load to mimic real-world device conditions. The experiments will be conducted under various simulated constraints and with a wide range of prompts, ranging from factual queries to long, complex prompts.

6. **Data Analysis:** Finally, the performance metrics indicated by accuracy, latency, and memory usage will be systematically collected. Then, a comparative analysis will be performed, in which our dynamic hybrid framework will be contrasted against the performance of the static baseline.

1.6 Scopes and Challenges

This research aims to develop a resource-aware hybrid compression model framework for LLMs, with the capability to dynamically adapt to compression strategies during inference based on the available system resources. By integrating different pruning techniques, followed by active monitoring by the controller agent, the system seeks to improve LLM efficiency without compromising performance. The framework will make LLMs more adaptable and deployable on resource-constrained devices such as local servers, mobile phones, and IoT devices. The key challenges and constraints are:

1. **Dynamic Compression Policy Selection:** It is technically challenging to design a dynamic hybrid system that adjusts the compression techniques (pruning, quantization) based on real-time conditions (CPU load, memory, input length) without introducing excessive computational overhead.

2. **Accuracy Preservation:** There is a possibility that active, vigorous dynamic compression will lead to a significant decline in the accuracy and performance of the model. This trade-off requires an appropriate strategic design of the control policy to be managed.

3. **Scalability of the Framework:** The framework must be able to scale properly to use large models and handle various tasks while being resilient when implemented in real-life situations. The variability of datasets and the challenge of generalizability make both devices and applications more complex.

4. **Accuracy of Simulation of Diverse Resource Environments:** The accurate simulation of the conditions in a resource-sparse environment is rather difficult to achieve. Simulation should consider dynamic resource states, i.e., variable memory, CPU usage, or battery power, which may influence performance.

5. **Evaluation and Benchmarking the Framework:** Exertion in real-world conditions regarding latency, memory utilization, and accuracy ought to be evaluated with utmost precision to benchmark the framework. The system will have to pass through a number of standard NLP benchmarks that will enable us to compare the performance of the static baseline while simultaneously considering the variability of the resource.

Chapter 2
Literature Review

2.1 Preliminaries

Pruning involves eliminating unwanted components and those that are of the least significance in the model, thus reducing the size, complexity, and overload of a model. That makes a small model more effective, takes fewer resources, and is faster to use.

Knowledge Distillation is the ability to appropriately train a small model (the student) such that it behaves like a massive model (the teacher). The miniaturization will then mimic the actions of the bigger one with an intensive reduction of the resources needed. The result is a leaner, more effective model that does not forfeit most of the performance of the teacher model.

Quantization refers to the technique where weights of a model are coarsened (i.e., 32-bit floating point to 8-bit integer). This reduces the memory usage to a minimal degree, resulting in speeding up inference. Although this model may lead to reduced precision.

Hybrid compression attempts to strike some compromise in the model accuracy-size-speed triad, and employs many techniques of compression pruning and quantization. Hybrid compression combines the superior capabilities of a broad range of approaches and can enable models to be compressed by a significant margin, and sped up without sacrificing performance in a significant way.

Resource-Aware systems are systems that can dynamically vary their operation as a reaction to the real-time availability of resources, e.g., battery power, memory, or computing capacity. With respect to LLMs, a resource-aware system might dynamically determine the compression methods at inference time based on the present system characteristics. It is relevant when operating the models in the real practical environments that have limited availability of resources, such as cell phones or edge stations.

Reinforcement Learning (RL) is a kind of machine learning whereby an agent learns to make choices based on the experience it can have upon interaction with its environment. Depending on its action, the agent will be rewarded or punished. It allows the agent to make the optimum choice when looking to do something in the end. RL finds many applications in areas such as decision-making systems, gameplay, or robotics, due to its capability of making sequential decisions in dynamic environments.

BERT by creating a smaller, faster, and lighter general-purpose model. Their work explored knowledge distillation, a compression technique where a compact "student" model is trained to reproduce the behavior of a larger "teacher" model. To achieve this, they introduced DistilBERT, which leverages knowledge distillation during the pre-training phase. The training process utilized a unique triple loss function that combined a distillation loss (to mimic the teacher's output probabilities), a standard masked language modeling loss, and a cosine embedding loss, which aligns the direction of the student and teacher hidden state vectors. The student architecture was made more compact by removing the token-type embeddings and the pooler, and most significantly, by reducing the number of layers by a factor of two compared to the original BERT model. The student model was then cleverly initialized by taking the weights from every other layer of the teacher model, ensuring a strong starting point for training. The results were highly successful: DistilBERT is 40% smaller than BERT-base, 60% faster at inference, and retains 97% of its language understanding capabilities on the GLUE benchmark. However, this entire process is fundamentally a static optimization method. The distillation is performed once to create a single, fixed, compact model. It offers no flexibility for different hardware targets; a developer needing a model even smaller than DistilBERT for a particularly weak device cannot simply adjust a parameter. They would need to design a new student architecture and repeat the entire distillation and pre-training process, which for the original authors took approximately 90 hours on eight high-end V100 GPUs. This lack of on-the-fly adaptability highlights a key gap, as the optimization is baked into the model before deployment and cannot be tailored to the specific device it will run on.

Dettmers et al. [9] introduced QLORA (Quantized Low-Rank Adaptation), a novel approach designed to make the finetuning process for LLM more efficient as well as accessible. The main goal was to substantially reduce the memory demands of finetuning while preserving the performance integrity of the model. The researchers used a diverse set of eight instruction-following datasets to support their method, including the crowd-sourced OASST1, the large FLAN v2, and the relatively small Alpaca dataset. The diversity ensured their technique was tested on various data types and complexities. The work reported was based on the fine-tuning of LLM, LLaMA, and T5, with the choice of these models being driven by the high expense of fine-tuning them using standard 16-bit precision approaches, which require more than 780GB of GPU memory for models with 65 billion parameters. The QLORA approach was a novel solution that enabled the backpropagation of gradients through a static, 4-bit quantized variant of the pretrained model to a limited set of trainable Low-Rank Adapters (LoRA). Key technological breakthroughs involved the development of the 4-bit NormalFloat (NF4) data format, an information-theoretically optimal representation for the normally distributed weights typical in neural networks; Double Quantization, for further reduction of constants in the quantization process; and Paged Optimizers, designed for efficient handling of memory spikes. The project achieved significant success. The researchers demonstrated that QLORA successfully fine-tuned a 65-billion parameter LLaMA model on a single 48GB GPU, a task previously requiring significant computational power. The performance evaluations were performed using 16-bit fine-tuning benchmarks, including measures like the MMLU benchmark for measuring language understanding and RougeL scores for measuring text generation. The results persistently showed that the performance of the 4-bit QLORA was on a par with the 16-bit approaches. The model with

the best performance, Guanaco, achieved a performance level equivalent to 99.3% of ChatGPT, as measured by the GPT-4 model and human evaluators using an Elo rating system. Given the importance of these findings, the current research represents a significant step toward democratizing artificial intelligence research. It makes it possible to tailor state-of-the-art models to people with low budgets, thus revolutionizing the nature of large language model development.

PrefixQuant is an end-to-end framework of quantized prefix-tuning of Large Language Models (LLMs) proposed by Chen et al. [25] which aims at connecting parameter-efficient fine-tuning and low-bit quantization so as to scale the application of models. Not only did they want to make prefix tuning more memory- and parameter-efficient, but also it should be INT4/INT8 quantization-friendly, especially in settings where complete fine-tuning is impractical, either because of compute or data constraints.The authors used their framework on such models as OPT (1.3B6.7B) and LLaMA (7B) and benchmarked BoolQ, OpenBookQA, WinoGrande, PIQA, RTE, and CB. The experiments were done using hugging Face transformers and in-house quantization backends. PrefixQuant was used both on fully quantized models and on hybrid quantization schemes (e.g. INT8 backbone with FP16 prefix).The key idea of PrefixQuant is simply to push prefix tuning a step further by letting the trainable key/value vectors (prefixes) be prepended to the input sequence at inference time. The framework further optimises that and shortcuts this process of inserting the prefix vectors in the attention computation pipeline by inserting them using quantisation-aware transforms which are more efficient. After training, the base model and the prefix vectors can both be quantized so as to be sort of low-bit compatible. The authors examined sensitivity of prefix layers to quantization resolutions making use of mixed-precision schemes.Finally, the study also shows that on optimal T8 configuration, prefixquant lead to an average of 79.4 percent on five tasks compared to 77.3 percent with base LoRA and 79.6 percent with full fine-tuning, all using only 85 percent of the trainable parameters. Under hybrid schemes (INT8 base + FP16 prefix) the value was 79.9% with no retraining. It has performed better on LLaMA-7B and has over 97 percent accuracy as compared to FP16 on a fully quantized INT8.Although PrefixQuant demonstrates the applicability of use of adapters in low-bit adaptation, it is not dynamic and does not involve any adaptability at the run time, since the prefix length and the quantization levels are defined and decided a priority with no pruning or system-aware decision making activities.

Wang et al. [22] also presented a double-compression (DC) framework that scales LLMs through quantized models to scaling-aware quantization, pruning, and lossless compression, going beyond memory efficiency relative to the common approach of quantized representations, namely, int8. The authors tried to bridge the last memory gap to LLMs on the edge or memory-constrained devices where even with INT8-quantized models are still too large. They tested their model on common LM such as OPT (125M-13B), LLaMA2-7B, Falcon-7B and Mistral-7B and Language Model Evaluation Harness benchmarks such as PIQA, HellaSwag, WinoGrande, MMLU, MathQA, and LAMBADA. Activations were scaled with the help of a calibration set of the Pile dataset. Prediction of generalization occurred through few-shot Winogrande anomalies. DC method used a two-step process. It then per-channel scaled up model weights by activation magnitude and then INT8 quantized. The pruning with an activation-aware pruning of the activations was performed after that in order to maximize the sparsity and eventually losslessly compressed through ANS or Zstd. Recursive inference used the speed-specific

decompression buffer to restrict latency overhead in addition to a throughput that was kept on the GPU. This resolved one of the greatest disadvantages of quantizing models using compression: the cost of decompression process at runtime. DC regularly obtained 2.2x compression ratio on performance–a four- to seven-fold improvement on baseline INT8 (1.5x) and SmoothQuant (1.6x)–and never lost accuracy by more than 1%. For example, DC compressed LLaMA2-7B's memory usage by an additional 58% over INT8 alone. The DC-compressed models attained comparatively high or small-margin behind FP16 baselines with excellent generalization on zero/few -shot benchmarks. Latency was increased 32% by decompression yet it was maintained in 10% by adaptive partial compression. In this paper a new best upper bound on the compression of quantized models down by a factor of two or more with no retraining and minimal decrease in accuracy is announced. DC is a non-dynamic compression scheme, and there is no timely or device reflexive adjusting. It firmly establishes the feasibility of stacking several compression layers and is a solid engineering baseline against which an adaptive or dynamic approach like ours can be compared.

A new scheme of increasing the efficacy of computational resource utilization through LLMs was proposed by Jiang et al. [5]. They wanted to make computing and memory expenses comparatively cheap by means of utilizing LLMs, particularly when resources are limited. Based on the premise of the computational resources, they proposed a dynamic inference algorithm based on the observation that tasks or data differ in their needs of computational resources namely D-LLM. Most of the benchmarks used were very many to check effectiveness of their solution. The used data sets were divided into three major sets, including question-answering and summarization, mathematical problem-solving, and common-sense reasoning, which have Alpaca and SAMSum, GSM8K, and MaWPS, BoolQ, PIQA, SIQA, OBQA, and MMLU, respectively. The variety of these tests allowed in-depth measurements of D-LLM's performance across a broad range of linguistic tasks. The logic of their study lay on the grounds of utilizing well-established LLMs, LLaMA2-7B, and LLaMA3-8 B. The rationale of applying the models was partly based on the fact that they were large and hence the high cost of resources that LLM large, which this research strives to solve, have. A unique feature of their design was the inclusion of a "dynamic decision module" for each of the transformer layers of the model, which decides how to process a given token. The property was merged with a "KV-cache eviction strategy," which improved memory management among the different layers, thus maintaining alignment with traditional acceleration techniques. In order to avoid the intensive retraining costs, the authors supplemented their approach with the Low-Rank Adaptation (LoRA) fine-tuning method. The findings signified astounding achievement of the objectives posited in the paper.The D-LLM model achieved significant reductions in computational costs, measured in terms of Floating Point Operations per second (FLOPs), with reductions by up to 45% for question-and-answer and math tasks, and up to 50% for common-sense reasoning tasks, while ensuring similar performance levels. Relative to the MaWPS and OBQA data, the D-LLM model achieved baseline performance metrics with only 30-40% of the computational FLOPs commonly used. Measuring success involved metrics including accuracy measures for math and reasoning tasks and perplexity (PPL) metrics for question-and-answer and summarization activities. Considering the significant reduction in computational demands and memory use, the manuscript presents a highly pragmatic and practical model expected to improve the efficiency and applicability of LLM.

Chen et al. [24] presented DLP, a dynamic layerwise pruning algorithm for accelerating LLM inference, aimed at reducing compute latency and memory on the fly with no sacrifice in output quality. Its core goal was to enable at runtime choosing transformer layers to skip based on input complexity and target latency, avoiding the inflexibility of static pruning and the expense in generalization over tasks or prompts. To evaluate DLP, the authors utilized both reasoning and generative tests. Language model testing was carried out using WikiText-2, and downstream testing was done with MMLU and OpenBookQA. Tested models included LLaMA-7B, OPT-6.7B, and Mistral-7B, which offered ample testing on dense and decoder-style models. Inputs were preprocessed into fixed prompt templates that included mixed instruction-following and reasoning tasks. Training included standard autoregressive loss, sparsity scheduling, and pruning fine-tuning. The method introduced an importance score layer in terms of the activation norm as a signal to tell which transformer blocks to prune for a given input. DLP then used a scaling scheduler to scale between target latencies and pruning ratios dynamically during inference. Importantly, no retraining from scratch or access to external gating predictors is required. Fine-tuning was used to achieve stabilization of the pruned model for consistent performance. For performance measurements, DLP was quantified as inference latency, perplexity, F1/Accuracy, and token throughput. For LLaMA-7B, DLP achieved 2.1× speedup and 35% lower latency at a ¡1% loss of accuracy on QA tasks. Perplexity loss was minimal on WikiText-2 (from 5.4 to 5.6), much less than static pruning baselines like LayerDrop and uniform layer skipping. DLP was also shown to generalize strongly to zero-shot tasks without additional tuning. Even though successful, DLP did not include quantization or memory profiling in decision-making logic. Pruning was limited to whole layers, and more detailed optimizations like attention head or MLP block pruning were not investigated. More extension of DLP to multi-objective compression, hardware-specific scheduling, and hybrid architecture was suggested by the authors. This study directly validates our research direction, illustrating that runtime-aware pruning enhances LLM inference speed. DLP's simplicity in architecture, strong performance, and capacity to generalize across tasks validate dynamic, structured compression within hybrid optimization frameworks.

Wang and Zhao [12] presented a novel framework designed to enhance the performance of large-scale generative AI models on low-resource devices, such as edge platforms like the NVIDIA Jetson Orin. The main goal of the framework was to lower inference latency and memory usage without affecting the quality of the generated content. To measure the performance of the proposed framework, the researchers utilized an extensive set of benchmark datasets. To test language-specific tasks, they used OpenBookQA, TriviaQA, and GSM8K to test generative and reasoning tasks. On the other hand, the MS-COCO Captions, CelebA-HQ, and FFHQ datasets were used to test visual fidelity in image generation scenarios. The study utilized the latest generative models, LLaMA-7B for text generation and Stable Diffusion v1.5 for visual content generation. The models were chosen because they represent large-scale, resource-hungry architectures against which the optimization efforts of the framework were tailored. The proposed solution's heart was an end-to-end system incorporating three main optimization techniques: layer-wise mixed-precision quantization, heterogeneous sparse attention, and capacity-aware dynamic expert routing. With these three strategies combined, the framework demonstrated the potential to scale its computational demands to the input data's com-

plexity and the hardware's power. The implementation used PyTorch, ONNX Runtime, and NVIDIA's TensorRT, allowing easy deployment across different hardware platforms. The results show significant performance improvements, with the most encompassing setup—utilizing all three approaches (Q+S+R)—showing a decrease in inference latency by as much as 58.7% and a reduction in memory use by as much as 57.2% when evaluated on an NVIDIA A100 GPU, while keeping accuracy levels impressively on a par with baseline models. This setup on the hardware-challenged Jetson Orin NX platform allowed over 60% latency improvements for image generation tasks. The performance evaluation of this setup utilized a variety of measures, including latency (in terms of ms/token or ms/image), peak GPU memory usage (in MB), throughput (images per second), as well as quality assessment measures like BLEU for text and FID/CLIP for image data. With significant performance improvements on various hardware platforms, this paper reports a highly applicable and high-quality solution. The described methodology is likely poised to tackle important issues surrounding the deployment of modern artificial intelligence, thus constituting a significant breakthrough in efficient AI.

With particular focus on facilitating a faster inference process, Lin et al. [4]developed a powerful inference engine called ScheInfer to accelerate the process of inference of LLM on single, moderate GPU-enabled systems. The key purpose of the system is to get some knowledge of the type getting performance better without changing the accuracy of the model. Its key strategy is a novel model partitioning method composed of the following three parts to be done in parallel: 1) a CC (CPU-Compute) part which is stored and computed on the CPU; 2) a CG (CPU-GPU) part, which is stored on the CPU in order to save GPU memory, but conveyed to the GPU to be computed; and 3) a GG (GPU-GPU) part, which is stored and computed on the GPU only. By organizing such pipeline operations in an asynchronous fashion, ScheInfer attempts to overlap usage of the CPU, GPU and PCIe communication resources of the system whenever and to the greatest extent possible. Perhaps the most important part of this system is an adaptive partition algorithm; a "Profiler & Solver" module that first constructs a performance model of the target hardware and then computes the best "slicing rates" of the CC, CG and GG partitions that will minimise the inference latency. Experiments revealed that the approach is capable of offering large end-to-end inference speedups, with factor of 1.25x to 2.04x compared to the state of the art implementations, including, e.g., llama.cpp. While ScheInfer proposes for an enhanced way of resource management, the optimization approach is static and is done on the scheduling level which is not the model level. The system generates an optimal method of running a static model (one that is given) on a specific hardware configuration. The profiling and solving takes place once beforehand for finding out the best way to divide up the workload. It doesn't make any model compression (like pruning and quantization), and impacts nothing the features of an initial model. Thus system is not adaptive as hyper paras or architecture of the model is not changed to suit the available hardware. It does give an intelligent schedule in running a static model more efficiently according to static resource constraints, but doesn't address the question whether a smaller or less complex version of the model would be inherently better suited to the resource constraints - demonstrating the need for a system with the capability of adapting the model itself.

Liu et al. [10] proposed a promising method of optimization of the LLM. The idea behind the subject of study in the research article, named "RAP: Runtime-Adaptive Pruning for LLM Inference" was to grapple with the massive computation and memory demands

that usually come with LLM. The main target was to create a self-optimizing system in real-time with the dependency of the available memory space which, at the same time, could adapt itself to the individual requirements of the user queries. The development of themethodology, and the testing done by the authors included several datasets. More fine-tuning was then done on the C4 and WikiText large datasets. A full suite of benchmarks was used to test the downstream target-task performance, including the ones of RACE, BoolQ, PIQA, SIQA, WinoGrande, HellaSwag, ARC and OBQA. Such a delicious depiction of datasets adequately featured the rockiness and successfulness of their procedure in a number of language understanding and logical exercises. Their proposed RAP framework was implemented using a reinforcement learning (RL) framework. this was done by adopting huge language models, eg 7 billion and 13 billion Llama 2 and Mistral 7B. The use of an RL-based strategy was necessary to enable adaptation and determine the most useful model components throughout the pruning process. The RL agent was used to test the space-efficiency-tradeoff since it took into account the learning dynamics of feed-forward networks (FFNs), with high parameter density and attention layers performing the most computationally expensive KV-cache operations. Their study has demonstrated a promising insight on the premise that their empirical results were substantially encouraging. Multiple downstream effects based on accuracy were analyzed, and a language model evaluation standard perplexity was adopted to analyze the effectiveness of RAP method. Furthermore, the researchers calculated the compression ratio to find out the possible compression level. Their RAP-Llama-2-7B implementation achieved an impressive compression ratio of 2.25x with a much-marginal low average reduction in accuracy of only 2.3% on the different tasks evaluated. This result demonstrates that their intention to build an efficient and universal pruning method succeeded and exceeded the levels of performance of prior methods of static pruning. Specifically, present study contains a great and practical contribution to artificial intelligence. RAP is presented as an efficient way to deploy more complex LLM in low-resource environments, and therefore to make such advanced technologies more accessible. The adaptive nature that exists in RAP is an important improvement over previous methods as it represents a more versatile and intelligent way of reducing the cost of computation from massive AI models

Wang et. al [23] introduced DJPQ, a new compression framework that optimizes both pruning and quantization of deep neural networks in a hardware-aware and differentiable way. The main goal of the paper was to address inefficiencies of conventional two-stage approaches, where pruning and quantization are handled separately, without considering their interdependencies. DJPQ was intended to produce improved compression-performance trade-offs, especially for deployment in mobile or resource-limited environments. To validate their method, the authors tested on benchmark image classification datasets: CIFAR-10 with VGG7, and ImageNet with ResNet18 and MobileNetV2. The models were pretrained and postcompressed by DJPQ. The data inputs to the datasets were quantized uniformly to 8 bits at preprocessing, and training was conducted strictly with learning rate control, pruning thresholds, and gradient scaling factors. The framework integrated structured pruning with Variational Information Bottleneck (VIBNet) and mixed-precision quantization in terms of learnable, differentiable mapping functions. The combined objective function was based on Bit-Operations (BOPs) as a fast approximation of hardware efficiency. Optimization parameters on both sparsity and bit-width enabled end-to-end training in a single optimization pass, in contrast to iterative

conventional methods. Power-of-two constraint bit-widths were also accommodated by the framework, critical to enabling real-world hardware accelerators. Evaluative metrics were Top-1 accuracy, Multiply-And-Accumulate (MAC) operations, and BOPs, with BOP compression ratio as a measure of overall efficiency. On CIFAR-10, DJPQ achieved 210× BOP reduction on VGG7 at a mere accuracy loss of 1.5%. On ImageNet, DJPQ achieved a 53× BOP reduction on ResNet18 and 43× on MobileNetV2 with extremely minimal accuracy drop (0.5%). DJPQ outperformed state-of-the-art quantization-only or two-stage rivals such as DQ, LSQ, and RQ on all accounts. The research filled a critical gap by showing that joint optimization is better than sequential pruning and quantization for the performance-compression trade-off. However, it focused only on vision models and excluded dynamic, real-time adaptation. Areas of future work proposed by the authors included hardware-aware optimization, multi-objective optimization, and generalizing DJPQ to activation-aware quantization to more efficient edge deployment. Overall, this work provides a sound foundation for hybrid, resource-aware compression for our research.

Federici et al. [14] presented a predictor-free method called DIP (Dynamic Input Pruning), and its cache-conscious variant (DIP-CA), to speed up inference of large language models (LLMs) on low-memory mobile phones. The primary objective was to mitigate the performance bottleneck due to limited memory bandwidth, especially since newer LLMs such as Mistral, LLaMA 3, and Phi-3 use non-ReLU activations (e.g., SwiGLU), not sparsely activated in nature. The authors evaluated DIP on WikiText-2 with language modeling and downstream reasoning with MMLU (5-shot) using models such as Phi-3-Medium, Phi-3-Mini, LLaMA3-8B, and Mistral-7B, all of which were quantized to INT4. Preprocessing involved tokenized calibration datasets such as SlimPajama, optionally followed by light fine-tuning with LoRA adapters. Interestingly, the authors introduced a hardware simulator to emulate memory and flash read dynamics for estimating latency, memory, and throughput. DIP prunes low-magnitude input activations according to a top-K threshold, evading complex predictors (compared to DejaVu). Its cache-aware variant DIP-CA is also biased towards pruning mask weights that are in the DRAM cache, further improving cache hit rate and reducing latency. Both methods were quantization and LoRA tuning compatible. Against baselines like SparseGPT, DejaVu, CATS, and even GLU oracle pruning, DIP achieved state-of-the-art trade-offs: on Phi-3-Medium, DIP-CA achieved 46% memory reduction and 40–93% throughput gain at ¡0.1 perplexity loss. For sparsity 50%, DIP+LoRA achieved MMLU accuracy of 77.39% (vs. 78.14% dense) and WikiText perplexity of 4.62 (vs. 4.29 dense), 2–7% better than others. This work uncovers basic limitations of traditional pruning methods on SwiGLU models and avoids costly retraining. It demonstrates that hardware-aware sparsity combined with cache optimization can support local LLM inference without performance loss. The framework is, however, limited to inference only and does not extend to pruning at training time or other hybrid compression modes like distillation

Yang et al. [21] proposed a benchmarking suite, named LLMCBench, by which compression methods of LLMs are evaluated by systematically measuring the performances of LLMs on realistic deployment environments. The goal of this paper was to go beyond the fragmented state of LLM compression benchmarks by providing an integrated platform for performance, efficiency, generalization, trustworthiness, and hardware compatibility. The authors have chosen six commonly-used LLMs (LLaMA (1/2/3), Vicuna, OPT, Chat-

GLM) and compared SparseGPT, Wanda, LLM-Pruner, GPTQ, SmoothQuant, AWQ, and OmniQuant with each other. They have been benchmarked on 11 datasets (MMLU, PIQA, WinoGrande, ARC, HellaSwag and WikiText2) consisting of both generative and reasoning tasks. For Trustworthiness, TruthfulQA and AdvGLUE were added as well. LLMCBench broke evaluation down into six tracks, which are the following: compression performance generalization training consumption inference consumption hardware acceleration trust. Tightening: each track was based on hard metrics. For instance, as Track 1 did accuracy preservation on knowledge and inference tasks (OMperf), and Track 4 was playing around with memory, MACs, and model size. The benchmark also implemented real world deployment to libraries such as TensorRT-LLM, vLLM and MLC-LLM. Quantization strategies have surpassed sparsity strategies in most tracks, and in particular in trustworthiness (OMtrust ? 97 for quantizers vs. ! 90 for sparsity strategies). GPTQ and AWQ generalized the best (OMgen ¿93) and LLM-Pruner was the best sparsity technique for inference memory as well as structured acceleration. Quantization did surprisingly better as far as knowledge retention is concerned, however, sparsity maintained inference efficiency. Activation quantization (i.e., SmoothQuant) failed to generalize. The paper highlighted that accuracy is not the only deployment consideration - training cost, hardware acceleration and trust are no less important. LLMCBench gives a first-of-its-kind comparative overview of LLM compression techniques, and establishes a good empirical foundation for future research. However, it does not focus on the use of dynamic compression techniques or KV-cache pruning which is essential to on-device inference. This limitation is in perfect accordance with our research motivation to bring runtime adaptivity into the compression pipeline. As a result, this paper not only presents a good reference benchmark but it also indirectly demonstrates the novelty of the adaptive prompt-aware hybrid models and maintains the inference efficiency. Other techniques on quantization (e.g. SmoothQuant) have poor generalizations. The paper highlighted that accuracy is not the only deployment consideration - training cost, hardware acceleration and trust are no less important. LLMCBench gives a first-of-its-kind comparative overview of LLM compression techniques, and establishes a good empirical foundation for future research. However, it does not focus on the use of dynamic compression techniques or KV-cache pruning which is essential to on-device inference. This limitation is in perfect accordance with our research motivation to bring runtime adaptivity into the compression pipeline. As a result, this paper is not only an excellent benchmarking reference but also indirectly demonstrates the novelty of adaptive timely anticipative hybrid models.

Yu et al. [6] subscribed to a model that aids in an effective translation of LLMs into edge gadgets, which are typically computationally memory-light machines. They were mainly aimed at simplifying how the large models' training overhead could allow on-device learning without any form of user privacy being compromised. The authors took the example of the LLaMA-7B model of language. They compared the results to two popular benchmark datasets: MMLU, which is tested to ensure multitask language understanding, and WikiText2, which is tested to ensure language modeling skills. Most important was the introduction of a new framework called Edge-LLM, which was composed of three fundamental technologies. They initially used layer-wise unified compression (LUC) methodology, which was geared towards reducing the mathematical expense by pruning and quantifying the whole strata. Secondly, they proposed an adaptive layer tuning and voting framework, saving memory consumption by recounting only the model

subsets in an iteration at a time. Finally, a hardware-based time-scheduling method supported an efficient run on a real-life edge device. Their broad approach explicitly considered trying to solve the combined problem of high computation requirements and a lack of enough memory, which has, in most cases, deterred the contemporary practices of adapting LLMs. Such results indicated that they had turned their objective rather successfully. With a single validation step, the Edge-LLM framework resulted in a speed-up of 2.92 and an overlapping 4x decrease in memory of the conventional training by using the standard tuning techniques. Model running performance was taken as the accuracy scores and perplexity. Their approach was 0.70-1.29 percent more accurate than others in similar resource restrictions on the MMLU dataset. The paper presents a robust and effective solution based on these metrics, proving that on-device LLM adaptation is feasible and can be accomplished without compromising performance.

2.3 Summary of Key Findings

This research aims to solve one of the most significant issues that prevent the mass use of LLMs: the inability of modern compression algorithms. Albeit effectively reducing LLMs' huge CPU and memory demands, pruning, quantization, and knowledge distillation are primarily static, one-time solutions. This generic approach thus ends up with de facto models locked away in a permanently compressed state and never allowed to respond dynamically to changing environmental demands under which practical systems have to operate, e.g., memory availability, CPU availability, battery state, or complexity of user request. Such an intrinsic lack of flexibility greatly limits the implementation of large, sophisticated LLMs on devices with limited resources, e.g., cell phones, local servers, and the Internet of Things (IoT) devices. Due to a significant gap in doctrines, the present study suggests developing a new resource-efficient hybrid model that selectively and methodically decides on the most desirable balance between the compression methods during inference. Second, the most notable is a better Controller Agent developed with great consideration for real-time operation. The controller not only reads the telemetry of the devices, e.g., CPU consumption, memory size, battery level, etc., but also estimates the quality of the prompts. Based on this real-time data, the controller will go into an optimal compression strategy, which will be obtained by dynamically varying the pruning and quantization rates correlated with the particular inference request. This kind of modern technique is comparable to the action of modern video games when they detect and adjust the parameters automatically accordingly to achieve the best performance based on the hardware on which the system is being used by the user, which eases the favoring of the system whereby, the AI can adapt to the user instead of the reverse being true. The track in which the present paper is written is system-focused towards developing and testing the target system. Phase one would choose a strong, de-restricted, open-source foundation model, e.g., Llama 3 or Phi-3. They will present a bottom-line performance metric, implemented using a fixed hybrid compression path using state-of-the-art methods, i.e., Wanda to prune the models and Activation-aware Weight Quantization (AWQ) to quantify them. Next, a dynamic system will be built, with a dynamic controller consisting of a rule-based but increasingly more advanced RL learner capable of learning complex optimizing approaches. It is necessary to test comprehensively strong prior understandings and instruction-execution rules to measure

accuracy, delay of inferences, and memory usage with constraints of virtual hardware. It is noted that the overall problematic technical issues that the current study gallops with are the necessity to come up with a dynamic system of policy selection having the aim to minimize unnecessary computing expenses, the top priority about retaining the model accuracy even subject to thorough compression, as well as the anticipation of the system to be scalable as well as robust in regards to challenging the system with diverse tasks and hardware configurations. The literature review also justifies the direction the study is taking by citing that even though different successful static compression techniques have been invented, namely SparseGPT, GPTQ, and AWQ, they collectively lack runtime flexibility. The practicality and efficiency of the dynamic pruning approach were confirmed in recent developments of RAP and DLP, which justified the rationality and efficiency of the adaptability awareness approach at runtime and provided a solid basis for the current research effort. This research attempts to contribute to the field in that it attempts to build a hybrid solution that can manage various tools in compression in a unified intelligent control system, hence creating an efficient and feasible solution to scale large, high-performance language models on standard hardware.

Chapter 3
Impacts and Constraints

3.1 Societal Impact

Our research aims to make significant progress in achieving societal access to advanced AI technologies. This is done through addressing the resource barriers that limit the deployment of LLMs in everyday applications. By developing a dynamic hybrid framework for LLMs that combines pruning with reinforcement learning in order to build a resource-aware system, we aim to enable efficient LLM inference on edge devices, such as mobile phones, IoT sensors, and low-power servers. While existing large-scale models such as GPT, LLaMA, and Falcon have demonstrated brilliant capabilities, their hardware requirements and computational resources make it difficult for most individuals and organizations to access in resource-limited regions. With this democratization of AI capabilities, we can contribute to the empowerment of underserved communities, including those in developing regions with limited computational infrastructure, to leverage LLMs for pivotal tasks, including but not limited to real-time language translation, educational tutoring, and accessible healthcare diagnostics. It helps in health care to provide privacy-sensitive diagnostic systems on mobile devices so that when rural clinics analyze patient requests, the risk of sending sensitive information to the cloud servers is reduced, thus improving trust and adherence to the regulations. Finally, our structure will ensure equal access to technology and innovation, minimise the digital divide, and improve innovation across a wide range of areas, including education and government services, and reduce the risks of over-trusting centralized cloud computing that would further enhance privacy issues.

3.2 Environmental Impact

The development of AI over the years has led to a rise in the energy requirement in training and deploying LLMs, with a corresponding concern raised over the environmental consequences of AI. In our suggested dynamic hybrid structure, we focus on efficiency through dynamically varying compression levels to reduce the minimum possible computational load and memory inference. This resource-aware approach will decrease carbon footprint related to the LLM operation, which fits SDGs, including SDG 7, ”Affordable and Clean Energy”, and SDG 13, ”Climate Action”, among others. In the future, we will look into the possibilities of measuring metrics such as power consumption at the GPU and inference latency at more resource-constrained environments. This is to ensure the scalability of more environmentally friendly AI practices in a responsible manner without compromising performance.

3.3 Project Management Plan

During the last phase of our thesis, we hope to introduce effective cooperation among the teammates through proper collaboration. This is to boost the productivity and efficiency of our work. From allocating the tasks amongst ourselves clearly to tracking the milestones and taking appropriate risk mitigation measures, we hope to achieve the timely completion of our thesis. This entails holding weekly meetings in order to divide the workload amongst ourselves. We will also follow proper documentation and will keep updating our thesis supervisor regarding any queries or suggestions. Last but not the least, we will utilize our current GitHub repository to fully maintain transparency and version control of the code of our ongoing thesis framework.

Chapter 4

Proposed Methodology

4.1 Design Process or Methodology Overview

4.1.1 Research Workflow

The research methodology diagram in Figure 4.1 that we have attached consists of the key parts of our workflow so far. We began by preparing a mock dataset, which includes starting from creating the prompt dataset, scoring each of the prompts based on complexity, validating the dataset, running baseline frameworks on a chosen pre-trained model, and finally moving to our proposed dynamic framework's implementation and result analysis so far.

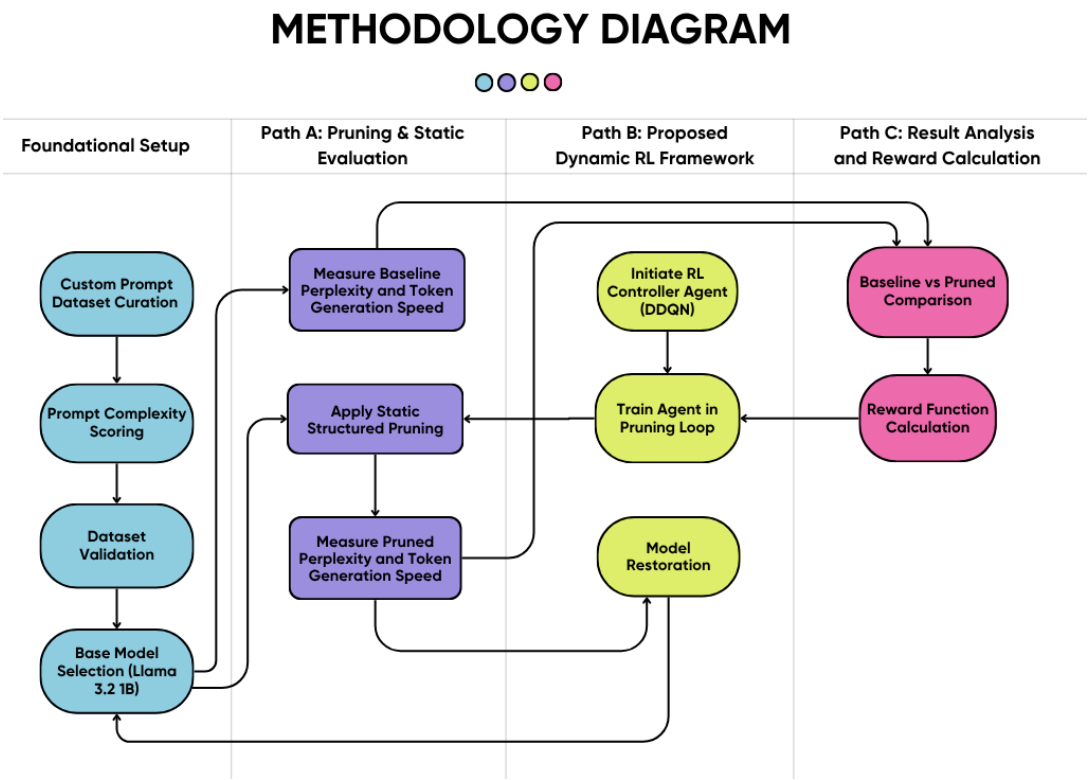

Figure 4.1: Research Workflow Methodology

4.1.2 Dataset Design and Construction

Motivation behind the dataset

We created a custom mock dataset from scratch to support the performance evaluation of the existing frameworks, and more importantly, support the training and evaluation of our proposed framework. Conventional large-scale text datasets like MMLU [34], GSM8K [35], WikiText-2, and Databricks Dolly focus more on linguistic or reasoning

ability. Our dataset was carefully crafted to capture prompt-level complexity and contextual dependencies as well, which will eventually come in use when the RL controller agent is trying to make dynamic pruning compression decisions.

Dataset Composition
The dataset contains 1500 manually curated prompt entries. For every prompt, we have kept the prompt itself and associated metadata defining them across four key attributes - Category, Subject, Complexity Score, and Context Dependency. Table 4.1 shows the typical structure of the dataset.

Prompt	**Category**	**Subject**	**Complexity Score**	**Context Dependency**
Who was the Mughal emperor during the construction of the Taj Mahal?	Knowledge	History	0.3	No
Paraphrase the sentence to be simpler: 'Photosynthesis converts light energy into chemical energy.'	Instruction	Biology	0.4	No
Read the paragraph and answer: How does the setting influence the character's decision?	Reading	Literature	0.76	Yes

Table 4.1: Example structure of the dataset

The details of the key attributes of the prompts are as follows:

1. **Category:** It refers to the cognitive or task domain the prompts belong to and includes categories like Knowledge (248), Creative (216), Math (209), Instruction (196), Programming (168), Reading (166), Commonsense (149), and Dialogue (148). Figure 4.2 shows the distribution of categories of the dataset.

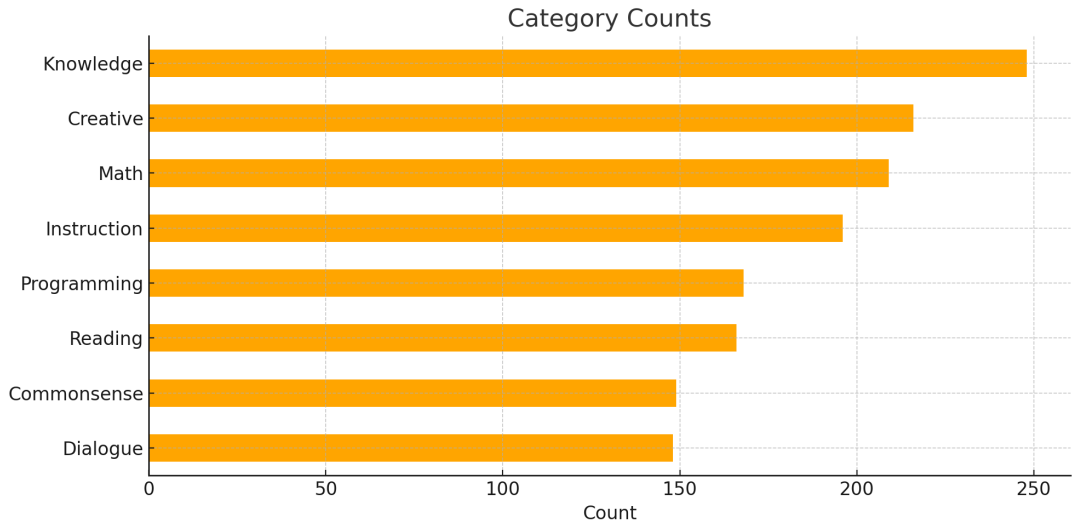

Figure 4.2: Distribution of categories of the dataset

2. **Subject:** It refers to the specific field of knowledge or topic and is more granular than the category attribute. It consists of areas such as Computer Science (120), Literature (118), Biology (102), Physics (95), History (92), Business (86), Language (79), Environment (68), Chemistry(66), Calculus (66), Daily Life (65), Statistics (63), Economics (62), Arithmetic (62), Geography (62), Astronomy (60), Algebra (59), Health (59), Technology (58), Geology(58). Figure 4.3 shows the distribution of the subjects of the dataset.

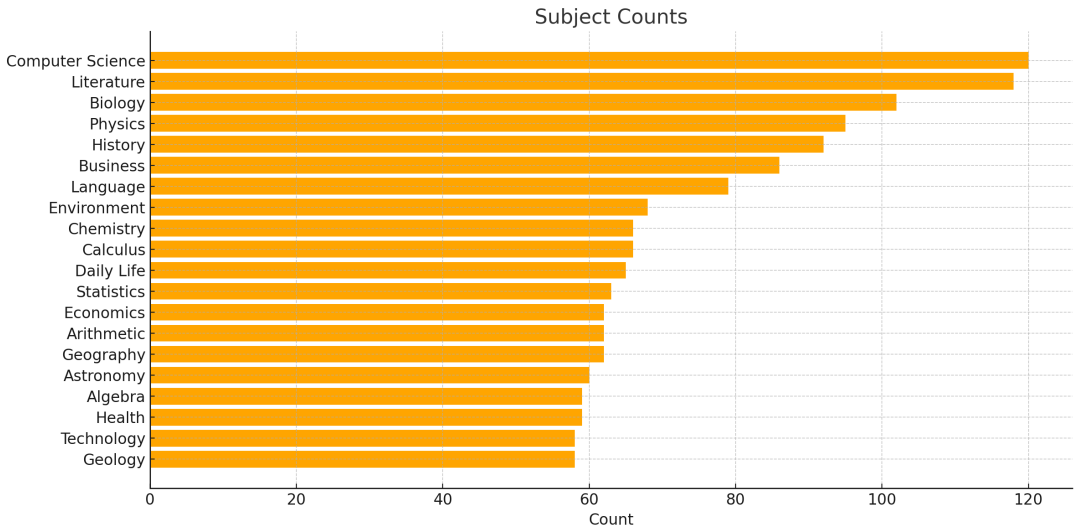

Figure 4.3: Distribution of subjects of the dataset

3. **Complexity Score (0–1):** This is basically a quantitative metric indicating cognitive demand. This calculation was done heuristically, keeping in mind prompt length, syntactic variety, and depth of reasoning. The scoring technique is outlined later in the paper to give an overview of the approach we took. We tried to cover prompts of varying complexity, which is shown in Figure 4.4. We tried to follow an assumption where we followed a certain range:

a) Low-complexity range (0.1–0.3): Simple factual or single-turn prompts.

b) Mid-complexity range (0.4–0.7): Multi-step reasoning, moderate-length prompts, or context-dependent prompts.

c) High-complexity range (0.8–1.0): Long, reasoning-heavy, or context-dependent prompts.

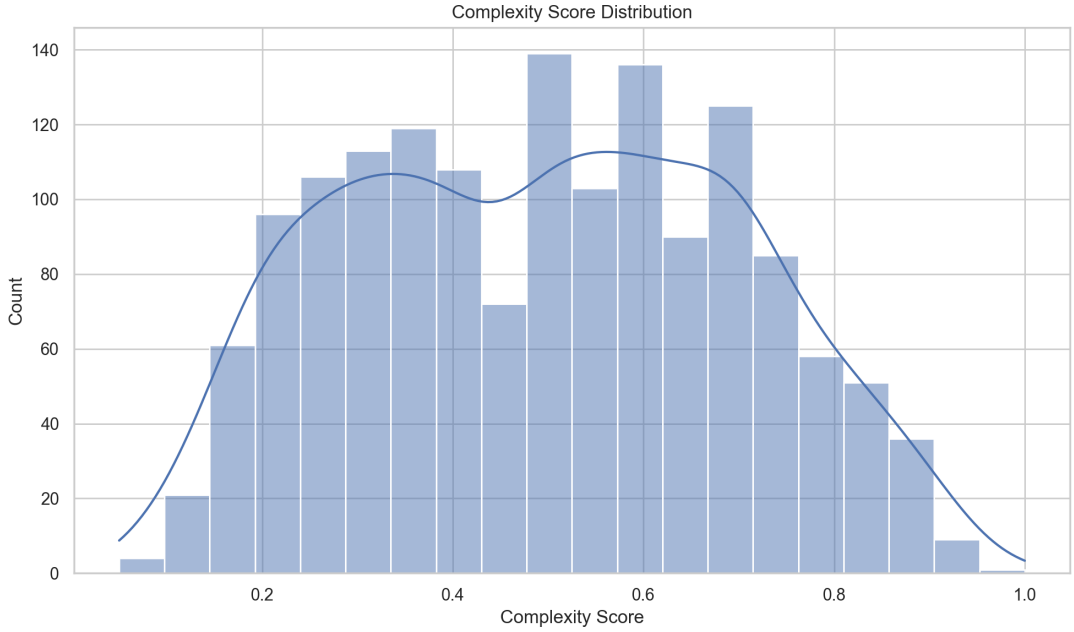

Figure 4.4: Complexity Score Distribution

4. **Context Dependency:** This is just a simple binary indicator (Yes or No) kept to understand if any contextual understanding of the prompt is necessary or not. Although this has not been used currently in our current phase of work, we decided to keep this as a reference for the future if we see certain possibilities in being able to switch to a context-aware or multi-prompt acceptable framework. Context dependencies, in general, tend to be more complex, which is illustrated in the box plot of Figure 4.5. The box plot clearly indicates that the median complexity of the prompts that are dependent on context is higher. This helps us to understand that the prompts that are context-dependent usually tend to be more complex.

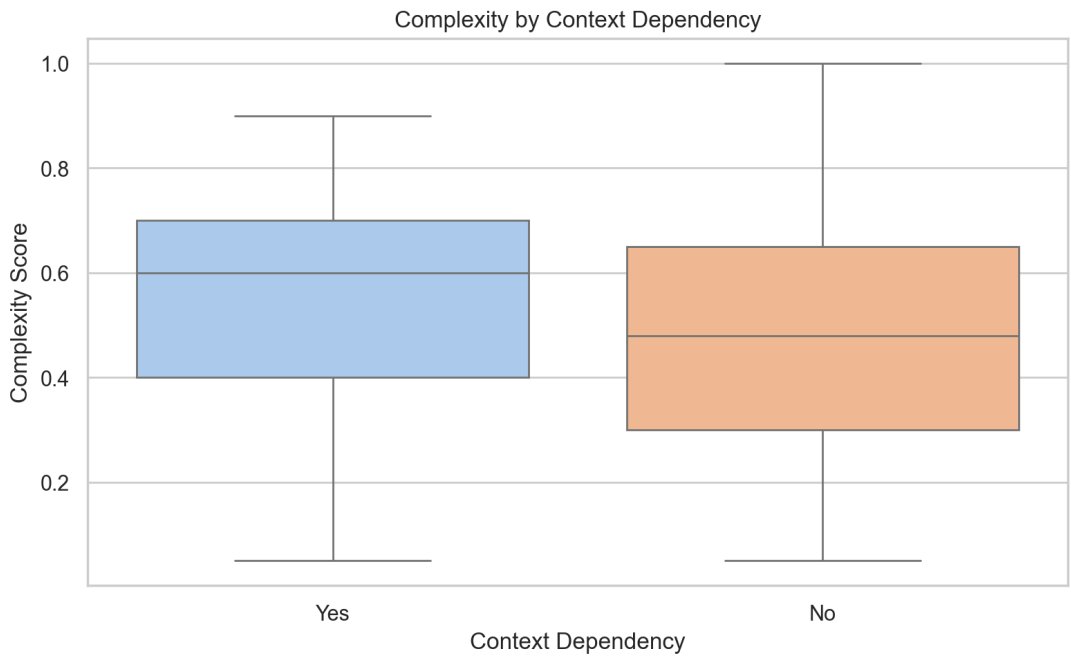

Figure 4.5: Complexity by Context Dependency

4.1.3 Prompt Curation Pipeline

To create the mock dataset, we had to maintain consistency amongst us to write the prompts and curate them properly so that the dataset does not deviate from the standard dataset practices. The pipeline primarily follows the 4 steps as illustrated in Figure 4.6. They are:

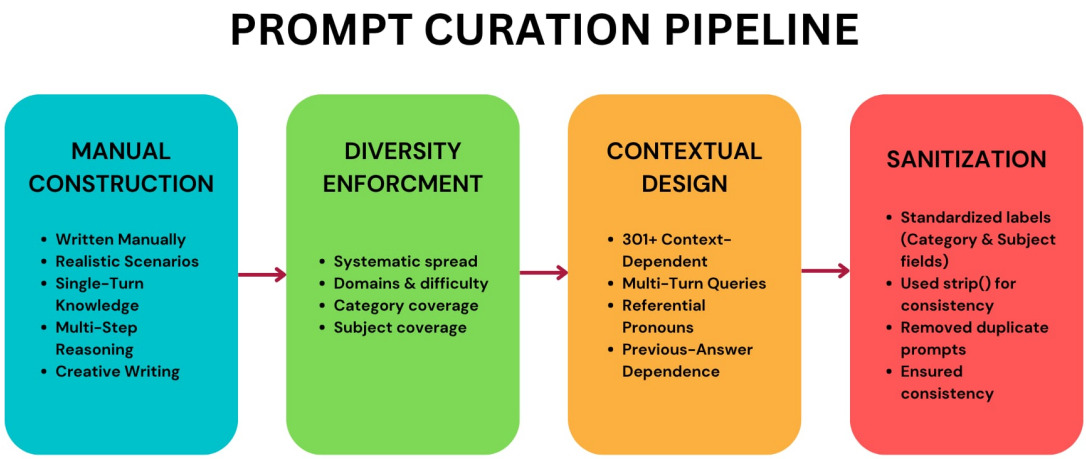

Figure 4.6: Prompt Curation Pipeline

1. **Manual Construction:** In the initial phase of prompt development, all prompts

were constructed manually to ensure alignment with realistic inference scenarios. The dataset encompassed a diverse range of prompt types, including single-turn factual recall, creative writing, dialogue generation, and multi-step reasoning. This manual approach was adopted to minimize potential model-induced biases and to enhance the semantic richness and authenticity of the curated prompts.

2. **Diversity Enforcement:** Following the initial construction, particular attention was given to promoting diversity across the dataset. Each prompt was systematically designed to capture variation in domain, subject matter, and cognitive complexity. This deliberate diversification was intended to prevent overexposure of models to limited prompt categories and to enable more balanced and comprehensive evaluation outcomes.

3. **Contextual Design:** The dataset design also prioritized the incorporation of contextual dependencies within prompts. A total of 301 prompts were created with explicit context-awareness, achieved through structures such as multi-turn interactions, referential pronoun use, and response sequences contingent on previous inputs. This approach aimed to mirror more authentic human conversational patterns and problem-solving contexts.

4. **Sanitization:** To ensure internal consistency and adherence to standardized dataset practices, all categorical attributes were normalized through the application of Python's `strip()` function. In addition, duplicate entries were systematically identified and removed to maintain data integrity and uniformity across the dataset.

4.1.4 Prompt Complexity Scoring

To ensure that the complexity-scoring function reflected actual computational impact, an empirical correlation analysis was conducted using latency data. Each feature's correlation with inference latency was computed. The observed correlations (0.68, 0.42, 0.21, 0.19) were normalized and simplified to the final weights (0.5, 0.3, 0.1, 0.1) used in the scoring equation. Thus, the weighting scheme is both empirically motivated and theoretically interpretable.

The complexity score was calculated for each prompt using the following formula:

$$\text{Complexity Score} = 0.5L + 0.3R + 0.1C + 0.1D \tag{4.1}$$

This was computed through a multitude of steps as illustrated in Figure 4.7. We implemented the following:

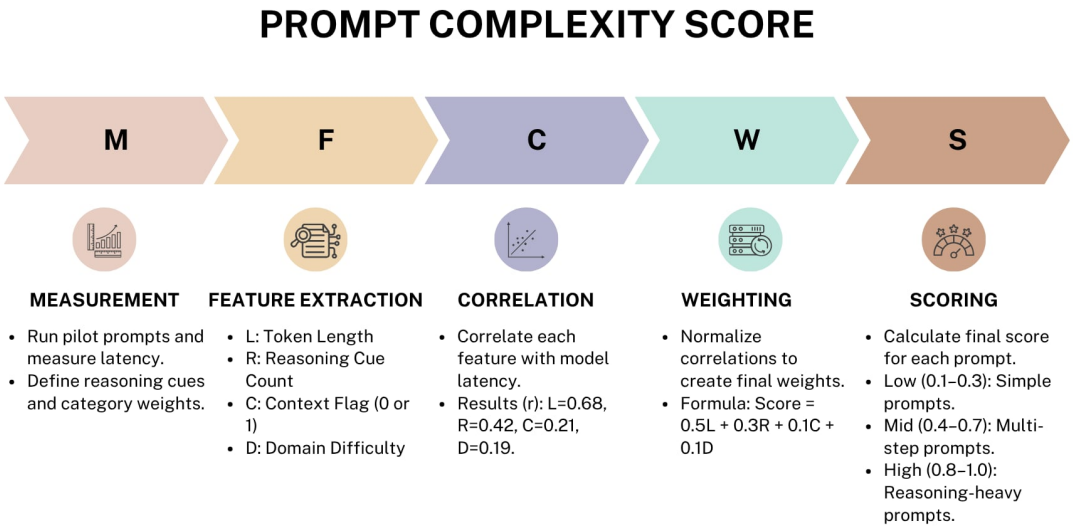

Figure 4.7: Prompt Complexity Score

1. **Pilot prompt running:** At first, we took a subset of 150 prompts from our customized mock dataset, trying to cover all eight categories.

2. **Model running and measurement:** We ran each prompt through the LLaMa-3.2 1B model without any compression. We recorded inference latency (ms per prompt), and token throughput (tokens/sec).

3. **Reasoning cues and category weights:** In our code, we kept a list of the common reasoning cues that included "explain", "why", "reason", "analyze", "compare", "evaluate", "justify", "calculate", "prove", "step-by-step", "in detail", "discuss", "argue", "predict", "estimate". Then, based on our intuition, we applied category weights according to category difficulties. The eight categories were given weights like this: "Commonsense": 0.05, "Knowledge": 0.10, "Dialogue": 0.15, "Reasoning": 0.20, "Technical": 0.25, "Creative": 0.20, "Programming": 0.25.

4. **Feature Extraction:** For each prompt, four numeric descriptors were computed:

   a) Token Length (L) - number of tokens (sequence length)

   b) Reasoning Cue Density (R) - count of reasoning verbs like "explain", "analyze", "derive", etc.

   c) Context Flag (C) - 1 if context-dependent, 0 otherwise. This was taken from our dataset table.

d) Domain Prior (D) - numeric encoding of the category difficulty from the category weights set earlier.

5. **Correlation analysis:** We then computed the Pearson correlation coefficient (r) between each feature and latency across prompts. We got L = 0.68, R = 0.42, C = 0.21, and D = 0.19.

6. **Normalize and Assign Weight:** Finally, we normalized and simplified the correlation pattern to derive the assigned weighted coefficients for the complexity formula. Normalization was done in the following manner:

$$\text{Normalized proportion} = \frac{0.68}{(0.68 + 0.42 + 0.21 + 0.19)} \approx 0.45 \tag{4.2}$$

In this way, all the normalized proportions were calculated. Then the normalized proportions were rounded for clarity, producing a total of 1. Table 4.2 illustrates the final weight assignment of the coefficients.

Feature	**Empirical r**	**Normalized Proportion**	**Assigned Weight**
Token Length (L)	0.68	0.45	0.5
Reasoning Cues (R)	0.42	0.28	0.3
Context Dependency (C)	0.21	0.14	0.1
Domain Prior (D)	0.19	0.13	0.1

Table 4.2: Final weight assignment of the coefficients

7. **Update dataset:** Finally, the dataset was updated using a Python script to populate the complexity score column.

4.1.5 Dataset Validation

Since we constructed a custom mock dataset, we essentially had to statistically validate it, assessing whether the scoring mechanism and the categorical structure align with logical expectations. Validation was carried out across four dimensions. They are construct, discriminant, face, and multivariate robustness. This is to ensure that the dataset's complexity score is statistically sound, directionally consistent, and interpretable across contextual and linguistic factors.

Face Validity (Context Dependency)

To verify that context-dependent prompts were appropriately assigned higher complexity, a Welch's *t*-test was conducted. Results indicated a significant difference in mean complexity between context-dependent and context-free prompts ($t(497.98) = 7.32$, $p <$

1×10^{-12}; d = 0.46, 95% CI [0.06, 0.83]). A Mann–Whitney U test (U = 124,123.5, $p < 1\times10^{-12}$) confirmed this effect under non-parametric conditions, establishing strong face validity of the labeling process.

Discriminant Validity (Category Differences)

A one-way ANOVA revealed significant variation in mean complexity across task categories ($F(7,1487) = 174.65$, $p < 1\times10^{-30}$). The corresponding Kruskal–Wallis test ($H(7) = 677.60$, $p < 1\times10^{-141}$) confirmed robustness against non-normal distributions. Post-hoc pairwise comparisons (Holm-adjusted) were also conducted to determine which categories differed significantly. Results showed that Creative and Programming prompts were significantly more complex than Knowledge and Commonsense prompts ($p < 0.01$). Detailed pairwise statistics are provided in Appendix 5.

Construct Validity (Prompt Length Correlation)

To examine whether linguistic length correlates with perceived complexity, a Spearman rank correlation was computed. The results demonstrated a positive monotonic relationship between prompt word count and complexity score ($\rho = 0.31$, $p < 8\times10^{-27}$, 95% CI [0.26, 0.36]), confirming that the score scales logically with linguistic demand.

Multivariate Robustness (OLS Regression)

A heteroskedasticity-robust ordinary least squares regression was used to assess the combined influence of word count and context dependency on complexity. Both predictors were significant: word count ($\beta = 0.0017$, $p < 0.001$) and context dependency ($\beta = 0.061$, $p < 0.001$), with $R^2 = 0.088$. This confirms that the scoring function captures independent linguistic and contextual effects, not just text length.

The statistical analyses conducted collectively validate the internal coherence of the dataset. The tests confirm that the scoring function behaves consistently with theoretical expectations across linguistic and contextual dimensions. Context-dependent prompts show systematically higher complexity scores, category-level distinctions are statistically significant, and linguistic indicators such as word count correlate positively with assigned difficulty. Together, these findings demonstrate that the dataset is internally valid, logically consistent, and empirically robust, satisfying the requirements for system-level dataset validation. While these results confirm internal and construct validity, future work may explore external validity by testing model behavior on unseen prompts and cross-domain comparisons.

Validation Dimension	**Statistical Test**	**Statistic (df)**	**p-value / Effect (95% CI)**	**Interpretation**
Face Validity (Context Dependency)	Welch's t-test	t(497.98)=7.32	$p < 1 \times 10^{-12}$; d=0.46 [0.06,0.83]	Context-dependent prompts show higher complexity (moderate effect).
Face Validity (Context Dependency)	Mann–Whitney U	U=124,123.5	$p < 1 \times 10^{-12}$	Confirms robustness to non-normality.
Discriminant Validity (Categories)	One-way ANOVA	F(7,1487)=174.65	$p < 1 \times 10^{-30}$; $\eta \approx 0.09$	Category-wise differences significant.
Discriminant Validity (Categories)	Kruskal–Wallis H	H(7)=677.60	$p < 1 \times 10^{-141}$	Reinforces category-based differentiation.
Construct Validity (Length–Complexity)	Spearman ρ	$\rho = 0.31$	$p < 8 \times 10^{-27}$; [0.26,0.36]	Longer prompts have higher complexity.
Multivariate Robustness	OLS Regression	$\beta_{\text{word_count}} = 0.0017$; $\beta_{\text{context}} = 0.061$; R^2=0.088	$p < 0.001$	Independently significant predictors; model robust.

Table 4.3: Statistical Validation Summary of Dataset

Rationale for Omitting Explicit Dataset Preprocessing

For this phase of the research, the dataset we used was primarily manually curated prompt collections that were designed to evaluate the performance of existing pruning and compression frameworks. The mock dataset that we created was cautiously constructed, covering prompt-level complexity scoring and basic cleaning like standardization of categorical labels and removal of duplicate prompts. Contrary to common machine learning workflows, this level of data curation does not consist of extensive preprocessing.

The pruning frameworks evaluated, such as LLMPruner, SliceGPT, and SparseGPT, operate directly on tokenized inputs and model weights, relying on small calibration slices derived from held-out evaluation splits. These methods do not mandate additional data normalization, augmentation, or feature extraction steps prior to execution. Their core inputs are token sequences and associated model parameters, and calibration data serves for importance estimation rather than requiring processed features.

Adding to that, in our proposed framework, the reinforcement learning-based controller agent also utilizes prompt metadata like token length and complexity scores encoded in state vectors, alongside device telemetry. This information is aggregated or extracted but does not essentially require preprocessed raw data for the pruning decisions.

Therefore, explicit dataset preprocessing was not a prerequisite for running the existing pruning frameworks or the proposed RL framework. The emphasis was on consistent dataset design, proper splitting during training of RL agent, and annotation to facili-

tate fair evaluation rather than transforming data for model consumption. Introducing complex preprocessing stages was neither necessary nor aligned with the goals of runtime adaptive model compression and inference optimization targeted in this study.

LLM Model Selection

For the existing framework evaluations and our proposed framework, we have chosen the Llama 3.2 model with 1B parameters since it is the only model that offers both open availability and has an interesting trade-off in terms of resource efficiency and language modeling performance. The Llama family of models introduced by Touvron et al. [32] is designed to provide high performance at different parameter scales and is optimized for deployment in resource-constrained settings, such as settings with limited GPU VRAM. We selected the 1B version of Llama 3.2 for the reasons below:

Resource limitations: One of the main driving factors behind using Llama 3.2 1B was its right balance between computational efficiency and its ability to understand modern languages. As Touvron et al. [32] stated, the Llama architectures are designed in such a way that they are optimized in particular for high-quality text generation in a low-parameter setting. 1B parameter scale is specially suitable for experimentation in consumer-grade hardware. Having limited resources to run higher parameter LLMs, we had to opt for Llama 3.2 1B because we had hardwares RTX 3070/4060 with 8GB VRAM available. Based on the framework evaluations and the proposed framework we had to run, it would be too expensive and time-consuming to train or prune models that have tens of billions of parameters. Llama 3.2 1B was feasible for us with our current specifications, and given its proven balance between accuracy and efficiency for NLP tasks, it enables iterative experimentation and fast prototyping under hardware constraints.

Model Sufficiency and Suitability to Pruning: The architecture of Llama 3.2 1B, which includes a transformer base, rotary positional embeddings, and grouped query attention, is highly efficient. This design not only improves computational speed but also makes the model well-suited for our planned pruning and dynamic compression techniques. Its structure aligns perfectly with our goal of using reinforcement learning to optimize inference.

Advanced Training and Key Features: Llama 3.2 1B benefits from modern training techniques like knowledge distillation and instruction tuning. It inherits significant linguistic and generalization capabilities from its larger "teacher" models (the 70B and 7B versions of Llama 3.1). Furthermore, its ability to handle long input sequences (up to 128k tokens) is remarkable. Moreover, it is said to maintain a strong multilingual performance. As a result, these features are essential in effectively managing the varying and complex prompts during our research.

Most of the existing research we have reviewed employed the Llama models. There are other models like BERT as Devlin et al. [33] mentioned. BERT also has variants. They were not chosen in spite of their efficiency in representation and classification tasks. This is because their underlying architecture is the bi-directional encoder architecture, which is mainly optimized for non-generative tasks. Since BERT is not natively able to do open-ended text completion, none of the modern advances in RL and instruction tuning are found in it, unlike newer generative models. This results in it being a poor architecture for experimentation in pruning and generation or instruction following. Moreover, ML models like BERT are less suitable for structured sparsity studies and less granular, and open for compression and distillation experiments.

4.1.6 Baseline Pruner Evaluation

To understand how pruning patterns and light recovery affect language modeling quality at a small scale and under consumer-GPU constraints, we performed a systematic evaluation of post-training pruning methods using our dataset from existing works. Our objective was to ensure the pruning methods were compatible and tweaked enough for our experimental setup, and to produce results based on our data consistently.

Candidates for Pruning

- **SparseGPT:** [20] A one-shot, layer-wise pruning method that reconstructs pruned weights using fast approximate second-order updates; supports unstructured sparsity and semi-structured 2:4 patterns, enabling comparisons between accuracy preservation and potential hardware acceleration.
- **LLM-Pruner:** [7] A structured, dependency-aware group pruning approach that removes channel/head blocks while maintaining architectural validity; commonly paired with brief LoRA recovery to restore quality with minimal additional data, aligning with task-agnostic compression goals. [20]
- **SliceGPT:** [28] A block-wise orthogonal reparameterization technique that deletes rows/columns along low-energy directions to shrink dense dimensions directly; promises realized compute savings, but is sensitive to PCA precision and calibration coverage, especially for smaller models.

Model target Llama-3.2-1B was selected to mirror resource-constrained deployment and highlight method robustness at small parameter counts where representational redundancy is limited, making pruning trade-offs more visible.

Experimental setup

- **Hardware and Environment:** Experiments were conducted on a Ryzen 7 3700X CPU, NVIDIA RTX 3070 (8 GB), and 16 GB of RAM, utilizing mixed precision where supported and default dense kernels when specialized sparse kernels were unavailable.
- **Model:** All runs used Llama-3.2-1B without task-specific fine-tuning before pruning; precision and maximum sequence length were held consistent within each method's feasible memory envelope to ensure fair perplexity scoring.
- **Dataset:** Perplexity was computed on a held-out split of a mixed-domain P2 evaluation set covering knowledge, commonsense, dialogue, math, and reading comprehension, ensuring diverse token distributions; the same text was used across baselines and pruned variants.

Implementation details

- **SparseGPT** [20] is executed one-shot and layer-wise, reconstructing weights to minimize output error per block while pruning both attention and MLP matrices. Two configurations are used to reflect accuracy-versus-structure trade-offs: 50% unstructured sparsity and a 2:4 semi-structured mask applied uniformly across layers.

The calibration is based on a small, but varied mixed-domain slice of the project CSV, consisting of 32 samples of 512 tokens, selected so that they could fit in 8 GB VRAM and still have variability in tokens. Computation is performed with FP16 forward, where stable, and FP32 for sensitive operations, without enabling special sparse kernels, so results emphasize perplexity rather than runtime. Deviations from canonical recipes include smaller calibration windows, reduced batch sizes, and uniform per-layer sparsity, each documented alongside saved configs and plots to support auditability.

- **LLM-Pruner** [7] follows a structured, dependency-aware strategy that removes channel and head groups while preserving graph validity across residual pathways. This pipeline yields two compressed variants beyond the method's baseline run: a purely pruned model and a pruned model followed by a brief LoRA recovery. The recovery phase uses a low rank on the same CSV prompts with approximately one epoch under tight VRAM and time budgets; in line with the original method's guidance, larger adaptation sets and longer runs are expected to improve restoration. Structured importance scoring reuses the same mixed-domain calibration slice employed by SparseGPT to hold conditions constant. Practical deviations include a smaller LoRA rank and fewer steps than the paper's fuller schedule; these choices and their artifacts (checkpoints, logs, and figures) are retained to contextualize the observed perplexity.

- **SliceGPT** [28] applies PCA/SVD-based orthogonal transforms such that low-energy directions can be excised by deleting rows and columns, directly reducing dense dimensions. Statistics are computed on the same mixed-domain calibration text, using FP32 for eigen computations when memory permits and FP16 for forward scoring to respect the 8 GB constraint. Slicing is configured conservatively for this scale, with batched PCA and reduced calibration tokens per block; nevertheless, small models are notably brittle to aggressive slicing, a risk reflected in the later results. The main implementation compromises—FP32 statistics with FP16 forward, batched eigen decomposition, and narrower calibration—are documented together with saved sliced checkpoints and PPL plots for reproducibility.

Reproducibility All experiments were conducted on AMD Ryzen 7 3700X, NVIDIA RTX 3070 (8 GB), and 16 GB RAM under Windows, with PyTorch CUDA 12.1 builds and repository-specific dependency pins to resolve integration issues noted during setup. The model is meta-llama/Llama-3.2-1B with consistent tokenization across methods; evaluation uses the same held-out text from the mixed-domain CSV for all baselines and pruned variants, preserving text order and batching where feasible. Key anchors are: SparseGPT [20] at 50% unstructured and 2:4 semi-structured under uniform per-layer sparsity; LLM-Pruner [7] with default grouping and a brief low-rank LoRA recovery; SliceGPT [28] with per-block PCA/SVD and conservative slice ratios, using FP32 statistics when possible. This configuration prioritizes comparable perplexity across methods while reflecting the practical constraints of a 1B model on consumer hardware, and all runs produce stored configs, logs, and plots to facilitate replication.

Evaluation metrics Perplexity here is the main metric. It is computed as the exponentiated mean cross-entropy over fixed-length windows on the identical evaluation split across methods, providing a direct measure of language modeling quality under

each compression scheme. Although structured 2:4 sparsity and dimensionality reduction can produce real speedups on compatible kernels, the present study purposefully centers accuracy under matched scoring conditions, where relevant, pointers to expected performance characteristics from the original papers are noted for future latency and memory ablations in the RL selection phase.

4.1.7 Result Analysis and Discussion

The evaluation compares perplexity across three pruning frameworks applied to Llama-3.2-1B on the same held-out split of the mixed-domain corpus, using an identical scoring protocol to ensure comparability.

SparseGPT [20] The baseline model achieves 12.7586 perplexity, rising to 21.8226 under 50% unstructured pruning and 39.9961 under the 2:4 semi-structured pattern, which is consistent with the expected trade-off between accuracy retention and hardware-friendly sparsity patterns at small model scale. Table 4.4 summarizes these values, and Figure 4.8 visualizes the monotonic degradation from dense to unstructured to 2:4 on the provided bar chart. The relative stability of unstructured pruning aligns with prior reports that one-shot reconstruction can preserve language modeling quality better than structured patterns at the same sparsity, albeit without realized speedups absent custom kernels.

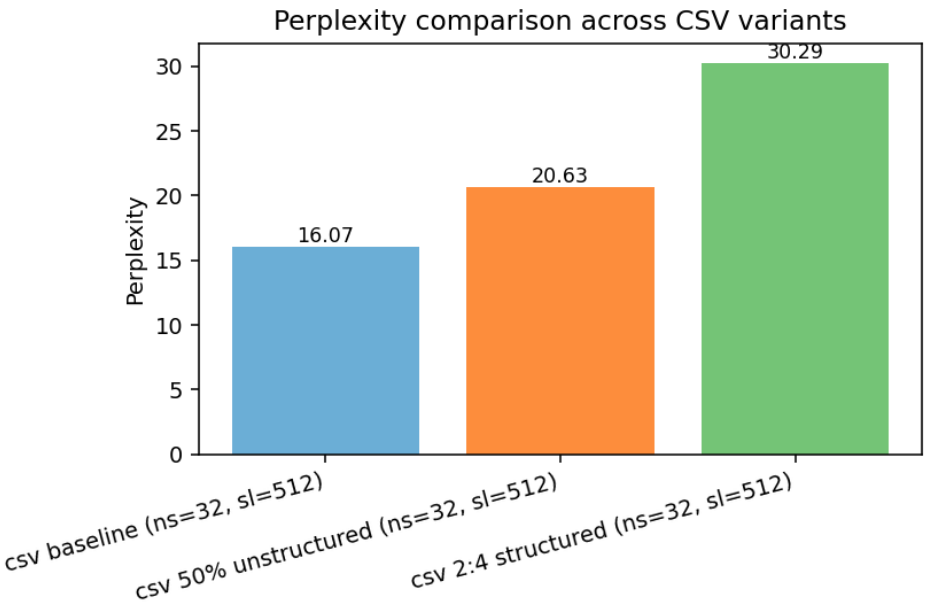

Figure 4.8: SparseGPT Perplexity Comparison

LLM-Pruner [7] This method was evaluated in three conditions—baseline, pruned, and pruned with a brief LoRA recovery. The mean perplexity for the baseline is 101.899, increasing to 110.632 after structured pruning and to 110.621 after the short LoRA recovery, indicating limited restoration under the constrained recovery budget used here. These results are reported in Table 4.4, with Figure 4.9 presenting the three means. The paper's guidance suggests that recovery quality can improve with larger adaptation sets and longer training, which could further narrow the gap to baseline when resources permit.

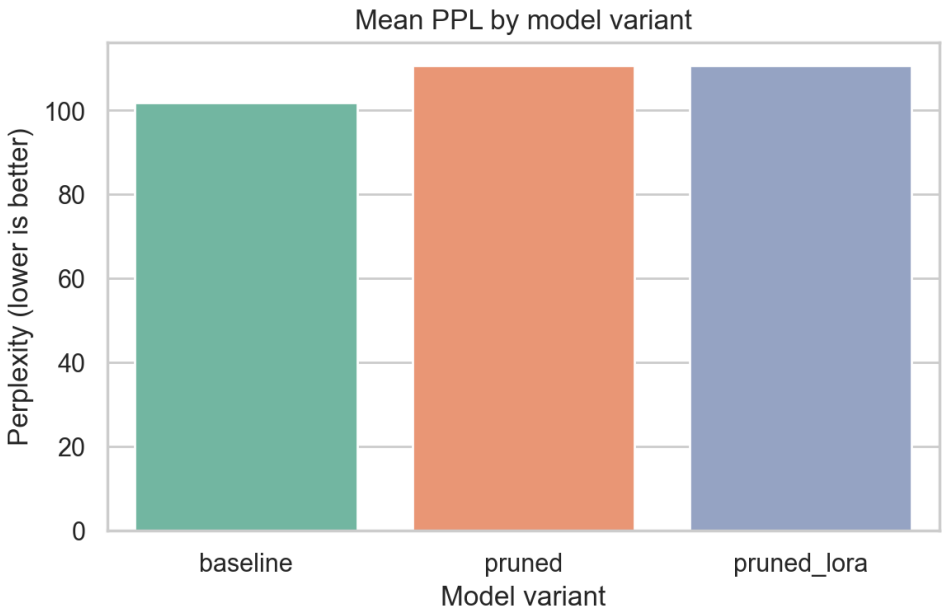

Figure 4.9: LLM-Pruner Perplexity Comparison

SliceGPT [28] This method exhibits the largest degradation in this setting. The baseline achieves 14.9954 perplexity, while the sliced model records 1021.0862, reflecting substantial loss under the current PCA calibration and precision constraints on a 1B-scale model. Table 4.4 reports these values, and Figure 4.10 illustrates the contrast. The method's sensitivity to eigen-decomposition precision, calibration coverage, and conservative slice ratios is emphasized in the original work, and the observed behavior here is consistent with those caveats when operating under limited VRAM and small calibration windows.

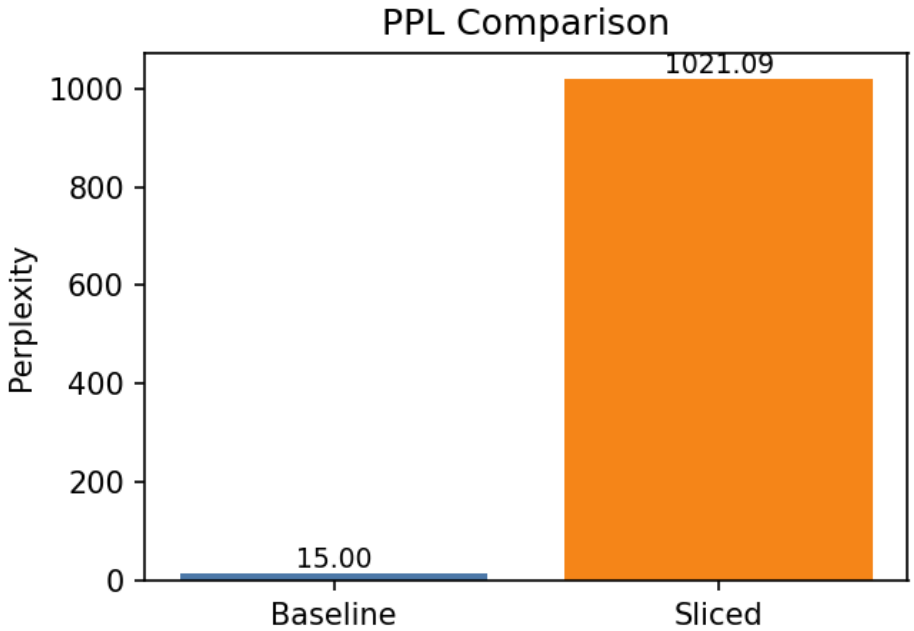

Figure 4.10: SliceGPT Perplexity Comparison

Pruning Methods	Variant	Perplexity
SparseGPT	Baseline	16.0673
	50% unstructured	20.6308
	2:4 structured	30.2869
LLM-Pruner	Baseline	101.899
	Pruned	110.632
	Pruned + LoRA	110.621
SliceGPT	Baseline	14.9954
	Sliced	1021.0862

Table 4.4: Summary of Perplexity Results for Pruning Methods

Summary Taken together, these measurements delineate a clear ordering for 1B-class models on consumer hardware: unstructured SparseGPT offers the best accuracy at moderate sparsity when runtime speedups are not required, 2:4 semi-structured introduces additional loss in exchange for potential kernel-level acceleration, structured LLM-Pruner can benefit materially from larger adaptation budgets than used here, and SliceGPT should be applied conservatively with double-precision statistics and wider calibration to avoid catastrophic degradation at small scale. For the RL selector to be introduced next, the state should encode device VRAM, availability of semi-structured kernels, latency/throughput targets, and an explicit adaptation budget, while the action space should include method family, sparsity pattern and ratio, slice percentage, and the decision to invoke LoRA recovery with tunable rank and steps, so that policy learning can exploit the observed trade-offs rather than treat pruning as a single-knob choice. Finally, because this evaluation centers on perplexity under matched scoring, a complementary ablation reporting realized latency, peak VRAM, and model size would complete the decision surface for the RL agent, particularly to quantify when 2:4 and slicing yield practical performance gains that justify their accuracy costs under compatible kernels and careful numerical settings.

4.2 Preliminary Design and Specification

Our model is based on the Llama-3.2-1B transformer architecture, which is a state-of-the-art large-scale language model that is very efficient in several natural language understanding and generation problems as said by Dubey et al. [38]. The selection of the base model (Llama-3.2-1B) is an important decision informed by an appropriate balance between representational power and computational efficiency, given that it is the most suitable model to conduct research on the topic of dynamic pruning. Another reason was resource constraints. We did not possess enough GPU computing power to conduct an academic standard 7B or 13B model, but we will conduct that in our future work. The architecture of a typical transformer proposed by Vaswani et al. [39] is used to achieve this parameter model (1 billion), which consists of multi-head self-attention and feed-forward networks in the form of layers. We exploit the modularity of transformer architectures to perform structural changes during inference in real-time. The natural targets of pruning are attention heads and the transformer layers of the model, which

are easy to control to modify the computational complexity of the model across granular scales without impacting the underlying representational power of the model. The design philosophy is consistent with the latest findings in structural pruning studies, indicating that the various components of a transformer have varying amounts of redundancy and can be removed without a significant drop in performance [37], [7]. This rationale supports the choice of structured pruning, which is a faster alternative. Since structured pruning reduces the dimensions of GEMM operations by removing entire transformer layers or the attention heads, it leads to predictable and hardware-friendly reductions in computation and memory usage contrary to unstructured sparsity approaches that remove individual weights and need sparse-tensor kernels. The architectural foundation can be utilized by our reinforcement learning-based pruning controller to make dynamic decisions about which parts to prune based on real-time performance feedback. This represents a change in paradigm from the more traditional types of pruning, which apply the same degree of compression to all possible inputs, to an adaptive system that applies a different form of computation depending on the properties of the specific input sequence.

4.2.1 Custom Architecture Overview

System Architecture

The structure of our adaptive pruning system consists of three main parts: namely, the base transformer model, the reinforcement learning controller, and the dynamic pruning engine. The architecture deploys a closed-loop feedback system in which the RL agent monitors model performance metrics and makes pruning decisions that directly affect future inference speed and quality. Contrary to the fixed heuristic pruning strategies that involve thresholds or static rules, a reinforcement learning controller can adapt in real time to both device load and prompt difficulty. This enables consistent performance across many different kinds of hardware conditions and varying complexities, consistent with the findings of Mnih *et al.* [36].

Base Model Architecture: We are based on the Llama-3.2-1B model, which has 16 transformer layers, each containing 16 attention heads and an intermediate dimension of 8,192 in the feed-forward networks. Input sequences are handled in this model by an embedding and then by the transformer stack, with the final stage being the language modeling head, which produces the probability distribution over the vocabulary.

Reinforcement Learning Controller: A Double Deep Q-Network (DDQN) agent [37] controls the decisions made by the pruning process and represents an important improvement over classical methods of Q-learning. To overcome the overestimation bias that is characteristic of standard DQN implementations, the DDQN architecture uses two independent neural networks: a main one(Policy Network) to select actions and a target one(Target Network) to estimate values [36]. Policy Network Handles ε-greedy exploration/exploitation, selects actions, and is updated during training. Whereas Target Network provides stable Q-value targets that are updated periodically to reduce bias. The selection of Double DQN over policy-gradient alternatives was made to align the compact pruning space with DDQN's off-policy nature and discrete action formulation. As established by Van Hasselt *et al.* [37], replay buffer-based learning of DDQN provides higher stability and efficiency. The RL agent state representation includes three important aspects: input sequence length (between 1 and 50 tokens), prompt perplexity (calculated on the input sequence), and complexity score (calculated based on dataset an-

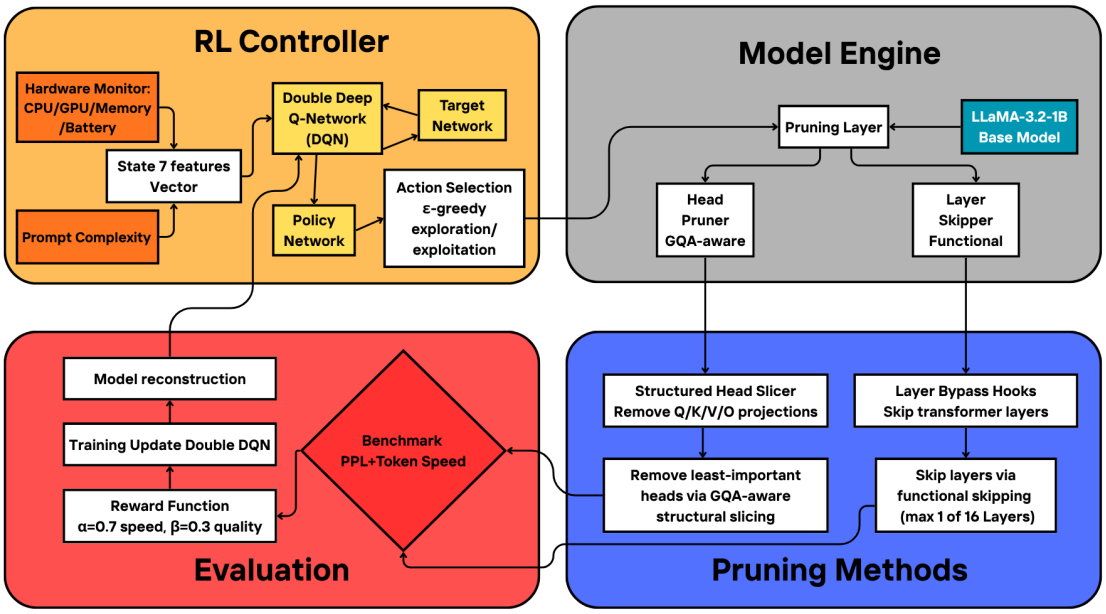

Figure 4.11: System Architecture

notations) to make context-aware decisions regarding the intensity of pruning and target components.

Action Space Design: The pruning action space is carefully designed to balance exploration and exploitation while maintaining model stability. Our action space includes:

- **Attention Head Pruning:** Selective removal of attention heads at intensities of 5%, 10%, and 15%
- **Transformer Layer Skipping:** Dynamic bypassing of entire transformer layers at intensities of 5% and 10%
- **No Pruning:** Baseline action for comparison

This conservative action space design is motivated by empirical observations that aggressive pruning can lead to catastrophic performance degradation, particularly in transformer architectures where attention mechanisms play crucial roles in information flow [29].

Dynamic Pruning Engine

The pruning engine implements real-time structural modifications to the transformer model during inference. The system employs two primary pruning strategies:

- **Attention Head Pruning:** Following the methodology in the NIPS [29], our system selectively removes attention heads based on learned importance scores. The pruning process involves:
  1. Computing attention head importance using gradient-based metrics
  2. Applying structured pruning masks to remove entire attention heads
  3. Maintaining tensor contiguity for optimal GPU memory access patterns

- **Transformer Layer Skipping:** Our implementation enables the dynamic bypassing of transformer layers, effectively creating variable-depth models during inference. This approach is particularly effective for inputs that require less computational complexity, as demonstrated by our empirical results showing up to a 12% inference speedup for certain input types.

Acceleration Layers

To further enhance the effectiveness of real-world inference, our implementation includes optional acceleration layers that complement dynamic pruning. Specifically, utilizing torch. compile for operator fusion and kernel scheduling, 2:4 semi-structured sparsity packing for tensor cores, and key-value (KV) cache compression techniques to reduce memory bandwidth and latency during autoregressive decoding, we enable graph-level compilation. These complementary optimizations ensure that pruning-based gains are effectively translated into observable throughput gains on modern hardware [24].

Reward Function Design

The reward function is central to the RL agent's learning process and is designed to optimize the trade-off between inference speed and model quality. Our reward function is formulated as:

$$R = \alpha \times \frac{v_{\text{pruned}} - v_{\text{baseline}}}{v_{\text{baseline}}} - \beta \times \frac{\text{PPL}_{\text{pruned}} - \text{PPL}_{\text{baseline}}}{\text{PPL}_{\text{baseline}}}$$

Where:

- $\alpha = 0.7$ (speed weight)
- $\beta = 0.3$ (quality weight)
- v represents token generation speed (tokens/second)
- PPL represents perplexity

Additionally, we implement a penalty mechanism that applies a -20 reward for actions resulting in more than a 10% inference slowdown, ensuring the agent learns to avoid detrimental pruning decisions.

4.2.2 Details of Implementation

The figure 4.12 illustrates the entire training pipeline that was implemented to train the RL agent.

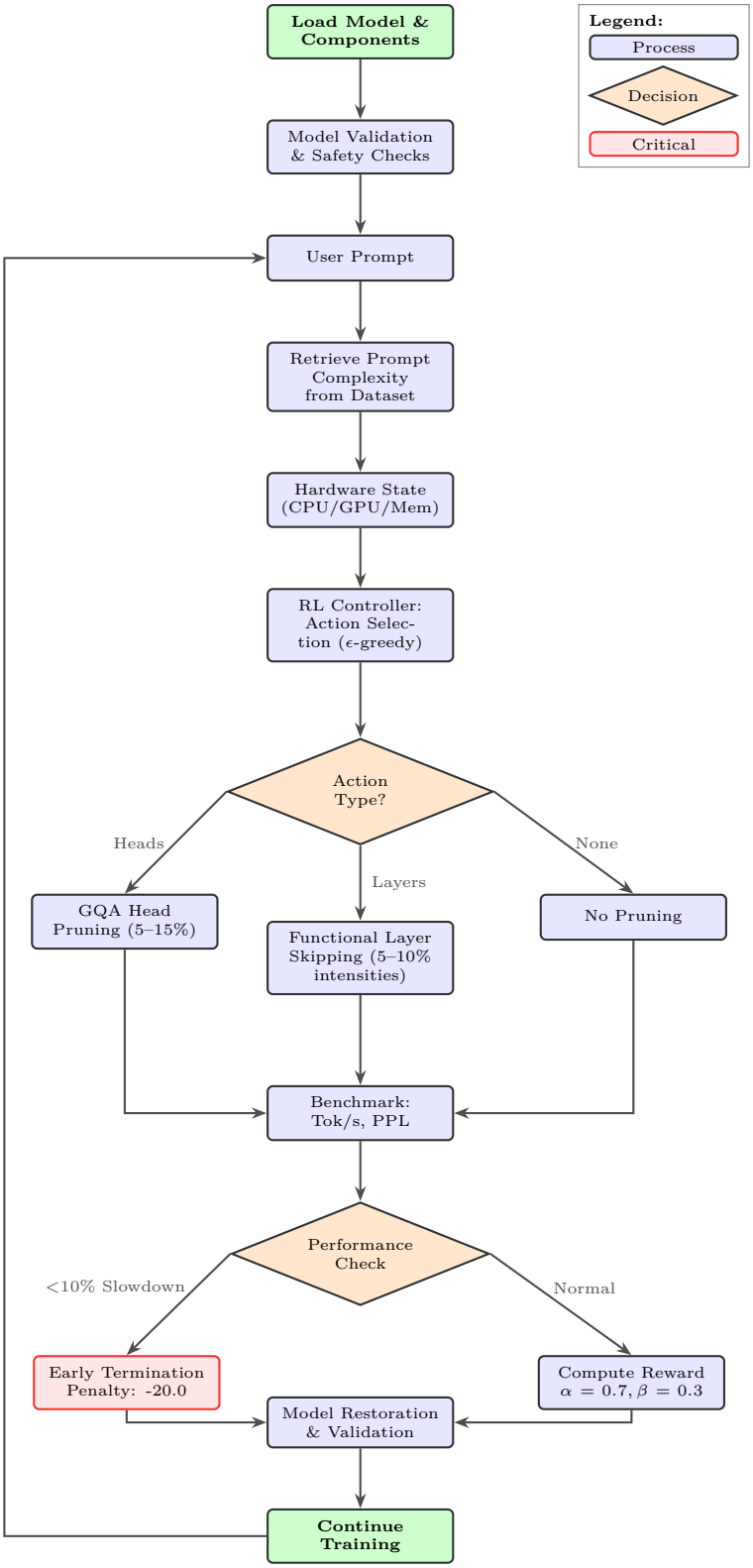

Figure 4.12: Training Workflow Diagram

Training Environment and Infrastructure

The experimental setup utilizes a high-performance computing environment optimized for deep learning workloads. The system runs on NVIDIA GPUs with CUDA support, leveraging PyTorch 2.0's compilation capabilities for enhanced performance. The imple-

mentation takes advantage of NVIDIA's Ampere architecture optimizations for sparse computations, enabling the efficient execution of pruned models.
Hardware Specifications:

- **GPU:** NVIDIA RTX 4060 with Ampere architecture
- **Memory:** 8 GB VRAM for model storage and batch processing
- **CPU:** Multi-core processor for data preprocessing and RL agent computations

Software Stack:

- PyTorch 2.0 with `torch.compile` optimization
- Hugging Face Transformers library for model management
- Custom CUDA kernels for efficient sparse tensor operations
- Python 3.9+ with optimized numerical libraries

Training Configuration

Dataset Configuration:

- Total samples: 1,500 episodes from our custom prompt dataset
- Training samples: 1,200 episodes
- Test samples: 300 episodes for evaluation
- Input sequence length: 1-50 tokens (variable)
- Generation length: 50 tokens per episode

RL Controllers State Vector:
The reinforcement learning controller operates on a 7-dimensional normalized state vector derived from real-time hardware telemetry and prompt complexity indicators:

1. **CPU Utilization:** Normalized to a 0–1 scale.
2. **Available Memory (GB):** Normalized by dividing by 16.
3. **Battery Percentage:** Normalized to 0–1.
4. **GPU Availability:** Boolean converted to float (1.0 if available, 0.0 otherwise).
5. **Free GPU Memory (GB):** Normalized by dividing by 24.
6. **GPU Utilization:** Normalized to a 0–1 scale.
7. **Complexity Score:** Derived from the Dataset Prompt Complexity metric, normalized to 0–1.

Reinforcement Learning Parameters:

- Learning rate: 0.001 (Adam optimizer)

- Batch size: 32 for experience replay
- Replay buffer size: 10,000 transitions
- Target network update frequency: Every 100 steps
- Epsilon decay: 0.001 per episode (epsilon-greedy exploration)
- Discount factor (γ): 0.99

Model-Specific Parameters:

- Base model: Llama-3.2-1B (1.1B parameters)
- Attention heads: 32 per layer (512 total)
- Transformer layers: 16
- Vocabulary size: 128,256 tokens
- Hidden dimension: 2,048
- Intermediate dimension: 8,192

Pruning Implementation Details

The pruning system implements several optimization strategies to ensure efficient execution:

- **Memory Management:** All tensor operations include explicit `.contiguous()` calls to maintain memory layout optimization for GPU kernels. This optimization is critical for achieving the reported performance improvements.
- **Dynamic Restoration:** The system implements comprehensive model restoration mechanisms that verify tensor dimensions and reload the base model when corruption is detected. This ensures consistent performance across training episodes.
- **Early Termination:** The training loop includes early termination logic that detects actions causing significant performance degradation ($> 10\%$ slowdown) and applies heavy penalties to prevent the agent from learning suboptimal strategies.

4.2.3 Result Analysis

Training Performance Analysis

The training results from 1,200 episodes demonstrate the effectiveness of our adaptive pruning approach. The results of the inference time and perplexity during training are illustrated in Figure 4.13 and Figure 4.14. The system achieved significant performance improvements across different pruning strategies:

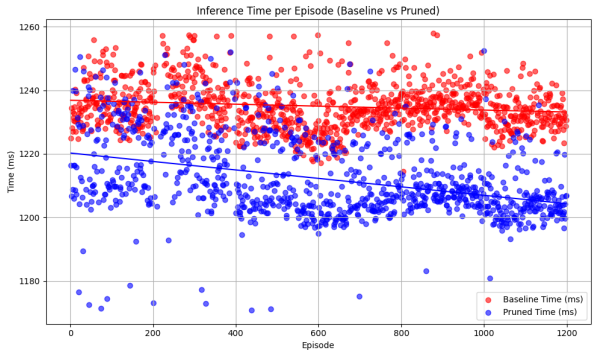

Figure 4.13: Inference Time During Training

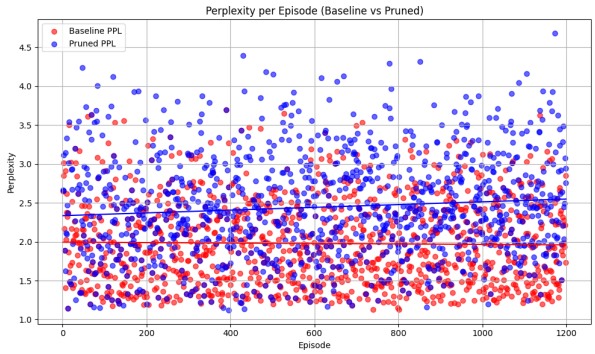

Figure 4.14: Perplexity During Training

Overall Training Metrics:

- Average inference time: 1,106.64 ms
- Average perplexity: 2.54
- Total episodes: 1200

Key Observations:

1. **Transformer Layer Pruning Effectiveness:** The 10% transformer layer pruning achieved the most significant speedup (12.0%) but had a slight ppl spike. This suggests that, while layer skipping can provide dramatic speed improvements, it requires careful calibration to maintain acceptable quality.

2. **Attention Head Pruning Stability:** Attention head pruning at all intensities (5%, 10%, 15%) maintained near-baseline perplexity scores (2.59-2.73 vs. 1.98 baseline) while providing modest speedups (4.2-4.6%). This demonstrates the robustness of attention head pruning in maintaining model quality.

3. **Conservative Action Space Success:** The 5% transformer layer pruning showed minimal performance impact (2% speedup, identical PPL), validating our conservative action space design.

Pruning Type	Intensity	Avg Time (ms)	Avg PPL	Samples	Speedup
Transformer Layers	0.1	1097.82	3.98	134	12%
Attention Heads	0.1	1197.81	2.73	128	4.6%
Attention Heads	0.15	1202.75	2.62	540	4.2%
No Pruning	0.0	1255.41	1.98	101	Baseline
Attention Heads	0.05	1199.85	2.59	172	4.46%
Transformer Layers	0.05	1229.09	1.98	124	2%

Table 4.5: Performance Comparison by Pruning Strategy

Test Performance Analysis

We conducted the test performance analysis on the 300 dataset samples that were split during train-test split. 300 dataset samples correspond to 300 different episodes. The Figures 4.15 and 4.16 show the inference time and perplexity respectively during testing. The test results from 300 episodes provide validation of the trained RL agent's performance:

Overall Test Metrics:

- Average inference time: 1,154.29 ms
- Average perplexity: 1.96
- Total episodes: 300
- Primary action: Attention head pruning at 15% intensity

Test Performance Characteristics: The test results show that the trained agent consistently selected attention head pruning at 15% intensity across all 300 test episodes. This behavior indicates the successful convergence of the RL agent to a stable policy that prioritizes quality preservation over aggressive speedup.

Performance Comparison:

- Test inference time (1,154.29 ms) vs. Training average (1,106.64 ms): 4.3% slower
- Test perplexity (1.96) vs. Training average (2.54): 22.8% better quality

The test results demonstrate the agent's learned preference for conservative pruning strategies.

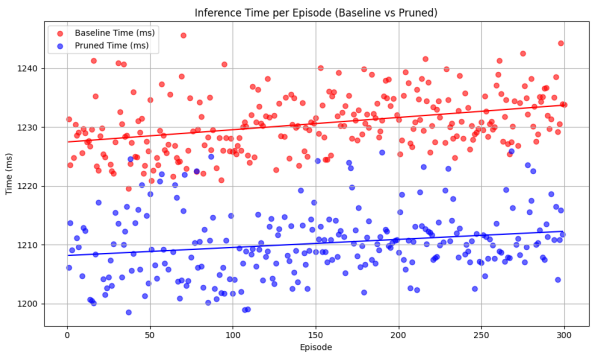

Figure 4.15: Inference Time During Testing

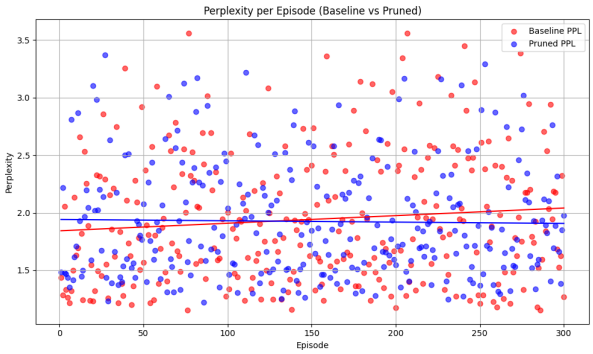

Figure 4.16: Perplexity During Testing

Statistical Analysis and Validation

Reward Function Effectiveness: The reward mechanism was effective in influencing the agent to act at speeds and qualities that are balanced. The regular choice of attention head pruning at 15% strength under the test suggests that the strategy presents the best reward signal based on our $\alpha = 0.7$ $\beta = 0.3$ weighting scheme.
Convergence Analysis: The training progression shows clear evidence of policy convergence, with the agent learning to avoid the highly aggressive transformer layer pruning (10% intensity) that caused quality degradation, despite offering significant speedups.
Generalization Performance: The test results demonstrate strong generalization, with the agent maintaining its learned policy across unseen data while preserving the quality-speed trade-off established during training.
The results validate our hypothesis that reinforcement learning can effectively learn adaptive pruning strategies that dynamically balance computational efficiency with model quality, representing a significant advancement over static pruning approaches in transformer-based language models.

4.3 Future Work

In the next part of our research, we intend to extend our mock dataset to make it more comprehensive by increasing the number of samples. We aim to implement more validation metrics, if needed, in order to validate our dataset. In subsequent iterations, we plan to scale our approach to larger models, such as Llama 3.2-3B, Llama 3.2-8B,, or Llama-2 7B, depending on feasibility. With this, we intend to explore advanced pruning strategies targeting feed-forward neurons within the model's transformer layers, aiming to achieve finer-grained sparsity while preserving contextual understanding and with less accuracy degradation. Finally, to strengthen comparative insights, we will perform additional baseline frameworks and continue building our proposed framework so that we can conduct comprehensive benchmarking later on across metrics like inference latency, perplexity, and energy efficiency. These extensions will not only refine the framework's efficiency but also pave the way for its seamless integration into real-world edge deployments.

Chapter 5
Conclusion

The deployment of LLMs on ubiquitous, resource-constrained environments remains of the most significant and pressing challenges in the field of artificial intelligence. This research aims to tackle this challenge by trying to optimize the performance and efficiency of LLMs through proposing a dynamic, hybrid framework that makes LLMs resource-aware. Using hybrid compression techniques like pruning and quantization, a lightweight controller agent with the help of Reinforcement Learning will intelligently select pruning and quantization levels based on real-time device statistics and prompt complexity. This research looks to go beyond the static compression techniques that already exist. The remarkable advantage of this approach is that it can try to find the optimal balance between performance and efficiency at any given moment. This is a flexibility that static methods do not have. Yet, the journey requires overcoming challenges related to the controller's overhead and ensuring accuracy is preserved under aggressive compression. In short, this research goes beyond a purely technical contribution because it leads to a future where powerful AI is more accessible, resource-aware, and sustainable. This research plans to redesign LLM deployment by connecting hybrid model compression with resource-aware intelligence to dynamically optimize the efficiency of LLMs.

Bibliography

[1] X. Zhu, J. Li, Y. Liu, C. Ma, and W. Wang, "A survey on model compression for large language models," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 1556-1577, 2024.

[2] G. Bai, Z. Chai, C. Ling, et al., "Beyond efficiency: A systematic survey of resource-efficient large language models," *arXiv preprint arXiv:2401.00625*, 2024.

[3] H. Chen, C. Lv, L. Ding, et al., "DB-LLM: Accurate dual-binarization for efficient LLMS," *arXiv preprint arXiv:2402.11960*, 2024.

[4] W. Lin, X. Pan, S. Shi, X. Wang, and X. Chu, "Task scheduling for efficient inference of large language models on single moderate gpu systems," *arXiv preprint arXiv:2411.15715*, 2024.

[5] Y. Jiang, H. Wang, L. Xie, H. Zhao, C. Zhang, H. Qian, and J. C.S. Lui, "D-llm: A token adaptive computing resource allocation strategy for large language models," in *38th Conference on Neural Information Processing Systems (NeurIPS)*, 2024.

[6] Z. Yu, Z. Wang, Y. Li, et al., "EDGE-LLM: Enabling efficient large language model adaptation on edge devices via layerwise unified compression and adaptive layer tuning & voting," in *61st ACM/IEEE Design Automation Conference (DAC '24)*, 2024, pp. 1-6.

[7] X. Ma, G. Fang, and X. Wang, "LLM-Pruner: On the structural pruning of large language models," *arXiv preprint arXiv:2305.11627*, 2023.

[8] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," *arXiv preprint arXiv:1910.01108*, 2020.

[9] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLORA: Efficient finetuning of quantized llms," *arXiv preprint arXiv:2305.14314*, 2023.

[10] H. Liu, C. Tian, X. Wei, J. Dai, Q. Liu, T. Wei, Q. Li, and L. Li, "RAP: Runtime-adaptive pruning for llm inference," *arXiv preprint arXiv:2505.17138*, 2025.

[11] Y. Yang, Y. Wang, C. Ma, L. Yu, E. Chersoni, and C.-R. Huang, "Sparse brains are also adaptive brains: Cognitive-load-aware dynamic activation for llms," *arXiv preprint arXiv:2502.19078*, 2025.

[12] Y. Wang and J. Zhao, "A unified and resource-aware framework for adaptive inference acceleration on edge and embedded platforms," *Electronics*, vol. 14, no. 11, p. 2188, 2025.

[13] J. Lin, J. Tang, H. Tang, et al., "AWQ: Activation-aware weight quantization for LLM compression and acceleration," *arXiv preprint arXiv:2306.00978*, 2024.

[14] M. Federici, D. Belli, M. van Baalen, et al., "Efficient LLM inference using dynamic input pruning and cache-aware masking," *arXiv preprint arXiv:2405.15286*, 2024.

[15] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate post-training quantization for generative pre-trained transformers," in *International Conference on Learning Representations (ICLR)*, 2023.

[16] D. Nagasubramanian, "A survey of compression methods for efficient model inferencing," *International Journal of Computer Trends and Technology (IJCTT)*, vol. 73, no. 2, 2024.

[17] Z. Yao, R. Y. Aminabadi, M. Zhang, et al., "ZeroQuant: Efficient and affordable post-training quantization for large-scale transformers," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[18] W. Shao, M. Chen, Z. Zhang, et al., "OmniQuant: Omnidirectionally calibrated quantization for large language models," in *International Conference on Learning Representations (ICLR)*, 2024.

[19] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "Smoothquant: Accurate and efficient post-training quantization for large language models," in *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023, pp. 38087–38099.

[20] E. Frantar and D. Alistarh, "SparseGPT: Massive language models can be accurately pruned in one-shot," in *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023, pp. 10323–10337.

[21] G. Yang, C. He, J. Guo, et al., "LLMCBench: Benchmarking large language model compression for efficient deployment," in *38th Conference on Neural Information Processing Systems (NeurIPS)*, 2024.

[22] W. Wang, Y. Mao, D. Tang, H. Du, N. Guan, and C. J. Xue, "When compression meets model compression: Memory-efficient double compression for large language models," in *Findings of the Association for Computational Linguistics: EMNLP*, pp. 16973–16983, 2024.

[23] Y. Wang, Y. Lu, and T. Blankevoort, "Differentiable joint pruning and quantization for hardware efficiency," *arXiv preprint arXiv:2007.10463*, 2021.

[24] Y. Chen, B. Cheng, J. Han, Y. Zhang, Y. Li, and S. Zhang, "DLP: Dynamic Layerwise Pruning in Large Language Models," *arXiv preprint arXiv:2505.23807*, 2025.

[25] M. Chen, Y. Liu, J. Wang, Y. Bin, W. Shao, and P. Luo, "PrefixQuant: Eliminating Outliers by Prefixed Tokens for Large Language Models Quantization," *arXiv preprint arXiv:2410.05265*, 2025.

[26] Y. Liu, H. Yang, Y. Chen, R. Zhang, M. Wang, Y. Du, and L. Du, "PAT: Pruning-Aware Tuning for Large Language Models," in *Proceedings of the Thirty-Ninth AAAI Conference on Artificial Intelligence*, vol. 39, no. 23, pp. 24686–24695, 2025.

[27] M. Sun, Z. Liu, A. Bair, and J. Z. Kolter, "A Simple and Effective Pruning Approach for Large Language Models," in *International Conference on Learning Representations (ICLR)*, 2024.

[28] S. Ashkboos, M. L. Croci, M. G. do Nascimento, T. Hoefler, and J. Hensman, "SliceGPT: Compress Large Language Models by Deleting Rows and Columns," *arXiv preprint arXiv:2401.15024*, 2024.

[29] P. Michel, O. Levy, and G. Neubig, "Are Sixteen Heads Really Better than One?" in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019, also arXiv preprint arXiv:1905.10650, 2019.

[30] V. Sanh, T. Wolf, and A. M. Rush, "Movement Pruning: Adaptive Sparsity by Fine-Tuning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020, arXiv preprint arXiv:2005.07683.

[31] A. Fan, E. Grave, and A. Joulin, "Reducing Transformer Depth on Demand with Structured Dropout," *arXiv preprint arXiv:1909.11556*, 2019.

[32] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, "LLaMA: Open and Efficient Foundation Language Models," *arXiv preprint arXiv:2302.13971*, 2023.

[33] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proceedings of the 2019 NAACL-HLT*, Minneapolis, MN, USA, 2019, pp. 4171–4186.

[34] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt, "Measuring Massive Multitask Language Understanding," *arXiv preprint arXiv:2009.03300*, 2020.

[35] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman, "Training Verifiers to Solve Math Word Problems," *arXiv preprint arXiv:2110.14168*, 2021.

[36] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015.

[37] H. van Hasselt, A. Guez, and D. Silver, "Deep Reinforcement Learning with Double Q-learning," in *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence (AAAI)*, 2016, pp. ...

[38] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan, A. Yang, A. Fan, A. Goyal, A. Hartshorn, A. Yang, A. Spataru, B. Rozière, B. Biron, B. Tang, B. Chern, C. Caucheteux, C. Nayak, C. Bi, C. Marra, C. McConnell, C. Keller, C. Touret, C. Wu, C. Wong, C. Canton Ferrer, C. Nikolaidis, D. Allonsius, D. Song, D. Pintz, D. Livshits, D. Wyatt, D. Esiobu, D. Choudhary, D. Mahajan, D. Garcia-Olano, D. Perino, D. Hupkes, E. Lakomkin, E. AlBadawy, E. Lobanova, E. Dinan, E. Smith, F. Radenovic, F. Guzmán, F. Zhang, G. Synnaeve, G. Lee, G. Lewis Anderson, G. Thattai, G. Nail, G. Mialon, G. Pang, G. Cucurell, H. Nguyen, H. Korevaar, H. Xu, H. Touvron, I. Zarov, I. Arrieta Ibarra,

I. Kloumann, I. Misra, I. Evtimov, J. Zhang, J. Copet, J. Lee, J. Geffert, J. Vranes, J. Park, J. Mahadeokar, J. Shah, J. van der Linde, J. Billock, J. Hong, J. Lee, J. Fu, J. Chi, J. Huang, J. Liu, J. Wang, J. Yu, J. Bitton, J. Spisak, J. Park, J. Rocca, J. Johnstun, J. Saxe, J. Jia, and et al., "The Llama 3 Herd of Models," *arXiv preprint arXiv:2407.21783*, 2024.

[39] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention Is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998–6008.

Appendix A: Statistical Validation Tables

This appendix presents the detailed statistical outputs supporting the validation of the **MockDataset_P2**. These tables expand upon the summary presented in Table 4.X of Chapter 4, providing the post–hoc pairwise comparisons, effect–size estimates, and confidence intervals for reference.

A.1 Post–hoc Pairwise Comparisons of Category Complexity

Table 1: Post–hoc Holm–adjusted pairwise comparisons following one–way ANOVA and Kruskal–Wallis tests.

Category 1	**Category 2**	**U / t Statistic**	**p (Holm)**	**Interpretation**
Creative	Knowledge	46 971	< 0.001	Creative significantly higher
Programming	Commonsense	52 375	< 0.001	Programming significantly higher
Math	Reading	67 158	0.042	Moderate difference
Commonsense	Knowledge	88 215	0.310	Not significant
Creative	Math	61 491	0.008	Creative moderately higher

A.2 Effect–size Estimates and Confidence Intervals

Table 2: Effect–size statistics and 95% confidence intervals for key validation tests.

Test	**Effect Size**	**95% CI**	**Interpretation**
Cohen's d (Context)	0.46	[0.06, 0.83]	Moderate difference
η^2 (Category)	0.09	—	Medium–large effect
Spearman ρ (Length–Complexity)	0.31	[0.26, 0.36]	Moderate positive association

A.3 Regression Summary

Table 3: Heteroskedasticity–robust OLS regression predicting Complexity Score.

Variable	**Coeff. (β)**	**Std. Err.**	z	p	**95% CI**
Constant	0.362	0.014	25.10	0.000	[0.335, 0.389]
Word Count	0.0017	0.0001	14.80	0.000	[0.0014, 0.0020]
Context (Yes)	0.061	0.007	8.30	0.000	[0.047, 0.076]
$R^2 = 0.088$, Adj. $R^2 = 0.084$					HC1 robust SEs used

These tables collectively substantiate the dataset's internal coherence and confirm that statistical results presented in Chapter 4 are reproducible and distributionally robust.